

실습환경 구축-1

01 실습환경 구축

02 웹과 HTTP 구조 이해

03 데이터 인코딩



학습 목표

- ✅ 보안 실습 환경의 구조를 이해하고, 로컬에 구축한다.
- ✅ 웹의 구조와 HTTP 통신 방식을 이해한다.
- ✅ BASE64와 URL 인코딩 및 디코딩 개념을 이해한다.



학습 내용

1. LAB 프로젝트 웹 서비스 구동 방법
2. 웹 서버와 애플리케이션 서버, 데이터베이스 간의 구조 및 요청 흐름 이해
3. HTTP 메시지 구조와 요청 메서드, 응답 코드의 역할
4. BASE64, URL 인코딩 및 디코딩 개념 이해



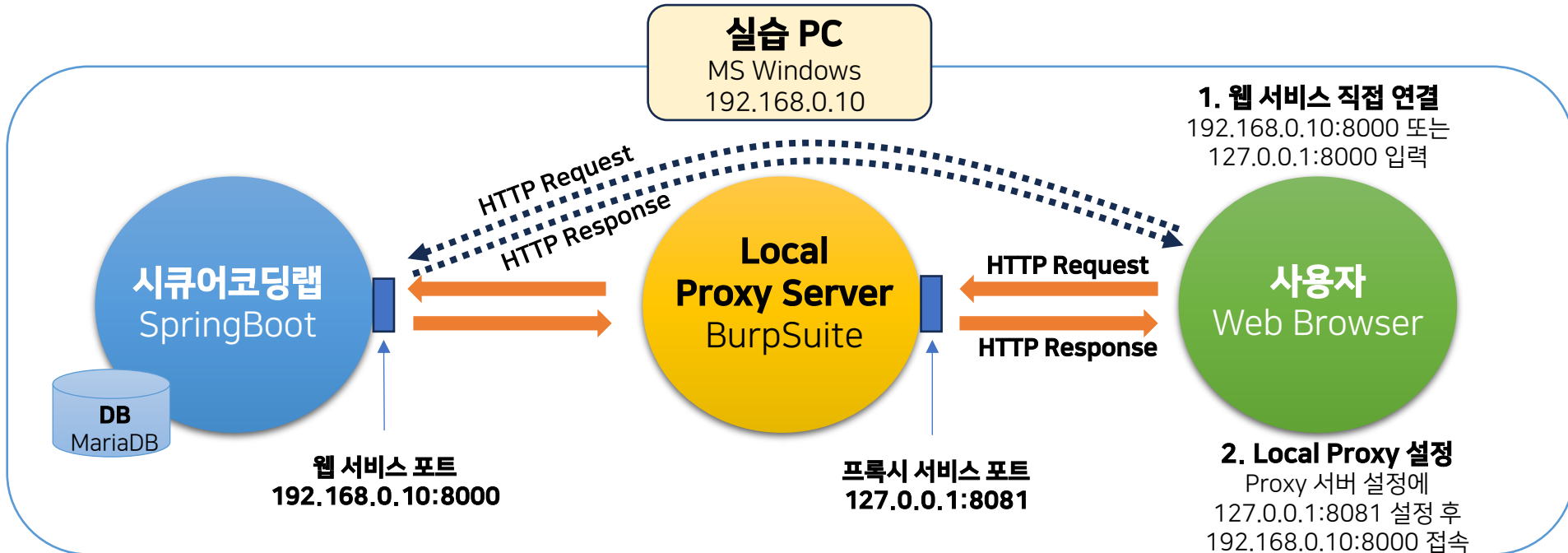
실습환경 구축-1

01 실습환경 구축

02 웹과 HTTP 구조 이해

03 데이터 인코딩

실습환경 아키텍처 이해



1. 웹 서비스 직접 연결

사용자는 웹 브라우저에서 직접 웹 서비스 포트에 접속하여 웹 서비스 사용 가능

2. 프록시 서버를 거쳐 웹 서비스 연결

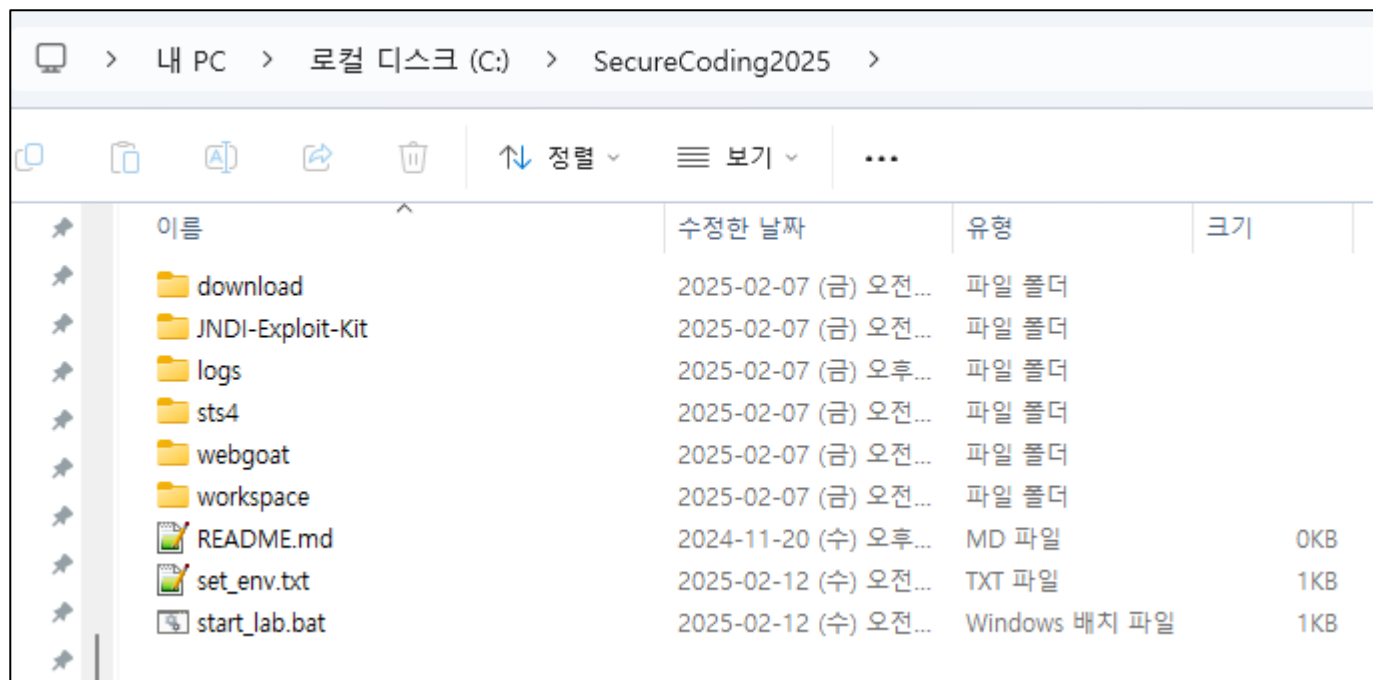
BurpSuite로 동작시킨 후, 웹 브라우저에 해당 프록시 서버를 등록하면, 모든 HTTP 요청과 응답을 프록시를 통해 주고받을 수 있음

실습환경 구축 - 웹 서비스 서버 설정 (1/8)

Step1. 배포된 LAB 파일 압축 해제

SecureCoding2025.zip 파일을 C:\W에 압축 해제

정상적으로 압축을 해제하면 아래 화면과 같이 폴더와 LAB 파일들이 생성됨



이름	수정된 날짜	유형	크기
download	2025-02-07 (금) 오전...	파일 폴더	
JNDI-Exploit-Kit	2025-02-07 (금) 오전...	파일 폴더	
logs	2025-02-07 (금) 오후...	파일 폴더	
sts4	2025-02-07 (금) 오전...	파일 폴더	
webgoat	2025-02-07 (금) 오전...	파일 폴더	
workspace	2025-02-07 (금) 오전...	파일 폴더	
README.md	2024-11-20 (수) 오후...	MD 파일	0KB
set_env.txt	2025-02-12 (수) 오전...	TXT 파일	1KB
start_lab.bat	2025-02-12 (수) 오전...	Windows 배치 파일	1KB

실습환경 구축 - 웹 서비스 서버 설정 (2/8)

Step2. 환경변수 설정

- 시스템 변수 3가지 추가

내PC > 오른쪽 버튼 클릭 > 속성 > 고급 시스템 설정 > 환경변수 로 들어가서 하단 '시스템 변수' 창에서 '새로 만들기' 선택

JAVA_HOME C:\SecureCoding2025\download\jdk-15.0.2\

MAVEN_HOME C:\SecureCoding2025\download\apache-maven-3.8.1\

MYSQL_HOME C:\SecureCoding2025\download\mariadb-10.5.10-winx64\

- 시스템 변수 Path에 3가지 경로를 제일 앞에 추가

'시스템 변수'에서 Path 환경변수 선택 > 편집 > 새로 만들기 선택 후 제일 위에 추가

%JAVA_HOME%\bin\

%MAVEN_HOME%\bin\

%MYSQL_HOME%\bin\

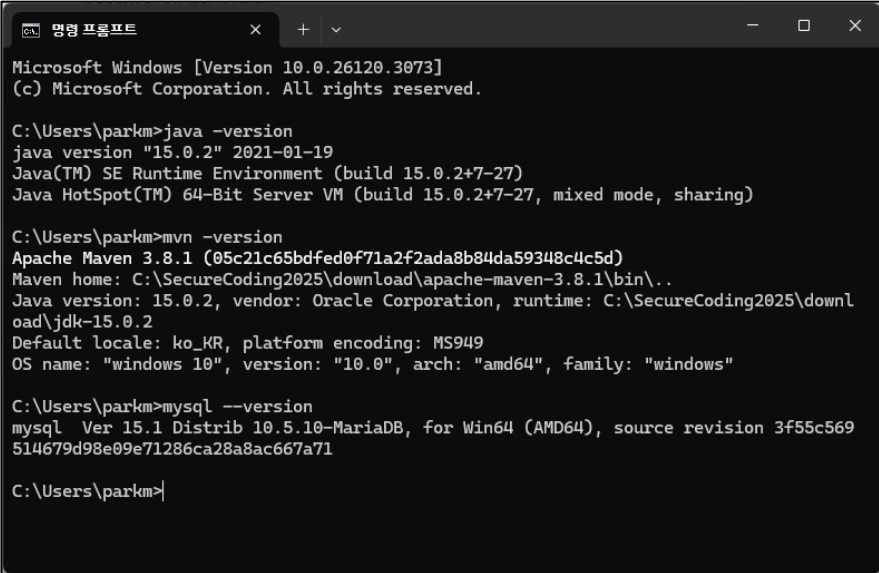
- 환경변수가 정상적으로 반영되었는지 확인

찾기 > cmd 실행하여 커맨드 창에서 정상적으로 버전이 출력되는지 확인

> java -version

> mvn -version

> mysql --version



```
Microsoft Windows [Version 10.0.26120.3073]
(c) Microsoft Corporation. All rights reserved.

C:\Users\parkm>java -version
java version "15.0.2" 2021-01-19
Java(TM) SE Runtime Environment (build 15.0.2+7-27)
Java HotSpot(TM) 64-Bit Server VM (build 15.0.2+7-27, mixed mode, sharing)

C:\Users\parkm>mvn -version
Apache Maven 3.8.1 (05c21c65bdfed0f71a2f2ada8b84da59348c4c5d)
Maven home: C:\SecureCoding2025\download\apache-maven-3.8.1\bin\..
Java version: 15.0.2, vendor: Oracle Corporation, runtime: C:\SecureCoding2025\downl
oad\jdk-15.0.2
Default locale: ko_KR, platform encoding: MS949
OS name: "windows 10", version: "10.0", arch: "amd64", family: "windows"

C:\Users\parkm>mysql --version
mysql Ver 15.1 Distrib 10.5.10-MariaDB, for Win64 (AMD64), source revision 3f55c569
514679d98e09e71286ca28a8ac667a71

C:\Users\parkm>
```

실습환경 구축 - 웹 서비스 서버 설정 (3/8)

Step3. 사용하고 있는 포트 확인

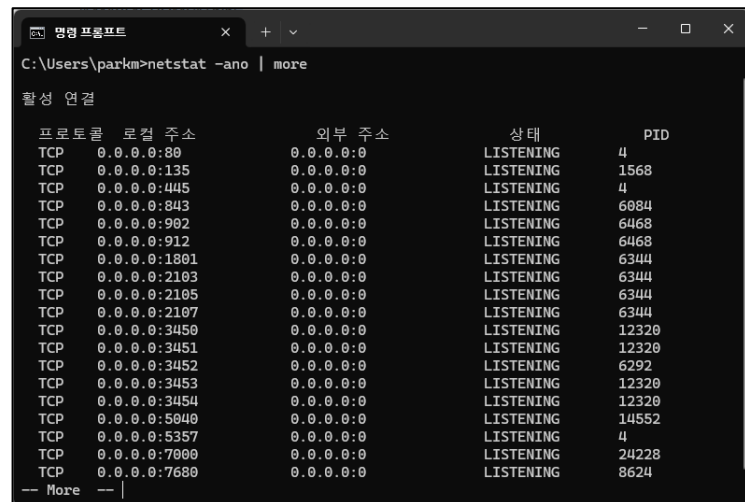
- 시스템에 이미 사용하고 있는 포트가 있는지 확인

cmd 창에서 아래 명령어를 입력하여, 실습에 사용할 포트를 이미 점유한 프로세스가 있는지 확인

3306, 8000, 8100, 8200, 8080, 8081

> netstat -ano | more

- 만약 LISTENING으로 보이는 포트가 있다면, 해당하는 프로세스를 찾아 종료
표시된 PID값으로 '작업 관리자' 에서 어떤 프로세스인지 확인 가능



```
C:\Users\parkm>netstat -ano | more

활성 연결

  프로토콜  로컬 주소      외부 주소      상태      PID
TCP        0.0.0.0:80    0.0.0.0:0      LISTENING  4
TCP        0.0.0.0:135   0.0.0.0:0      LISTENING  1568
TCP        0.0.0.0:445   0.0.0.0:0      LISTENING  4
TCP        0.0.0.0:843   0.0.0.0:0      LISTENING  6084
TCP        0.0.0.0:902   0.0.0.0:0      LISTENING  6468
TCP        0.0.0.0:912   0.0.0.0:0      LISTENING  6468
TCP        0.0.0.0:1801  0.0.0.0:0      LISTENING  6344
TCP        0.0.0.0:2103  0.0.0.0:0      LISTENING  6344
TCP        0.0.0.0:2105  0.0.0.0:0      LISTENING  6344
TCP        0.0.0.0:2107  0.0.0.0:0      LISTENING  6344
TCP        0.0.0.0:3450  0.0.0.0:0      LISTENING  12320
TCP        0.0.0.0:3451  0.0.0.0:0      LISTENING  12320
TCP        0.0.0.0:3452  0.0.0.0:0      LISTENING  6292
TCP        0.0.0.0:3453  0.0.0.0:0      LISTENING  12320
TCP        0.0.0.0:3454  0.0.0.0:0      LISTENING  12320
TCP        0.0.0.0:5040  0.0.0.0:0      LISTENING  14552
TCP        0.0.0.0:5357  0.0.0.0:0      LISTENING  4
TCP        0.0.0.0:7000  0.0.0.0:0      LISTENING  24228
TCP        0.0.0.0:7680  0.0.0.0:0      LISTENING  8624

-- More --
```

실습환경 구축 - 웹 서비스 서버 설정 (4/8)

Step4. start_lab.bat 실행

- LAB 실습에 사용할 데이터베이스(MariaDB)와 웹 서비스 개발환경(Spring Tool Suite 4) 실행 cmd 창에서 C:\WSecureCoding2025로 이동하여 준비되어 있는 시작 배치 파일 실행

C:\SecureCoding2025> start_lab

- DB 정상 구동 확인

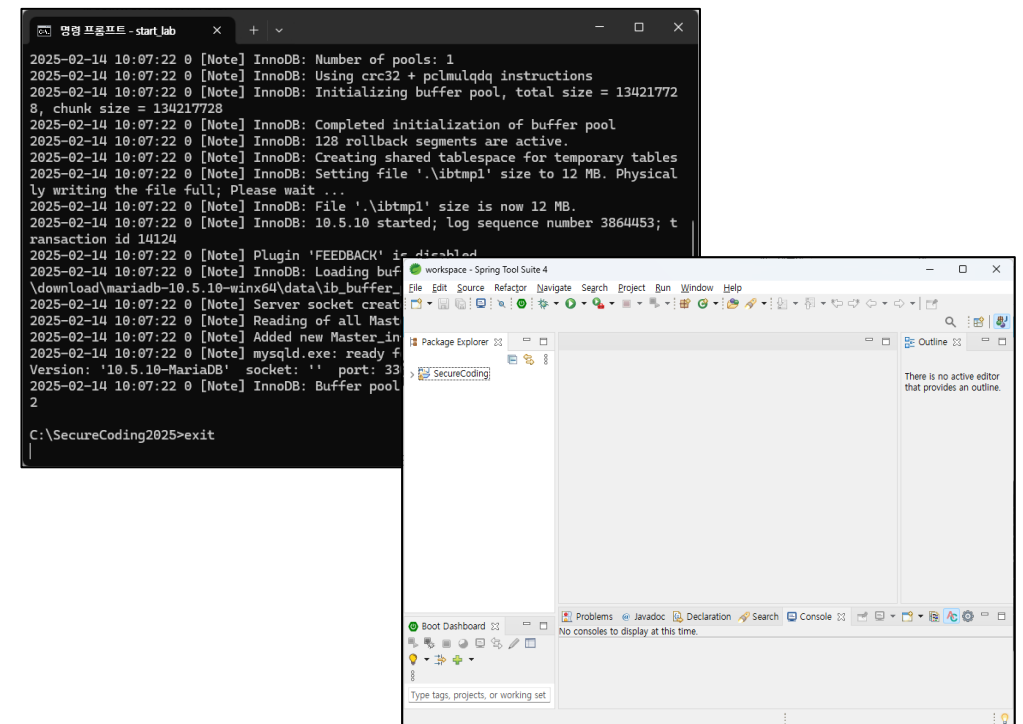
cmd 창에 나타난 로그 내용으로 정상 여부 확인됨

정상: [Note] mysqld.exe: ready for connections.

cmd 창은 최소화시켜놓기만 하고, 종료시키지 말 것

- Spring Tool Suite 4 정상 구동 확인

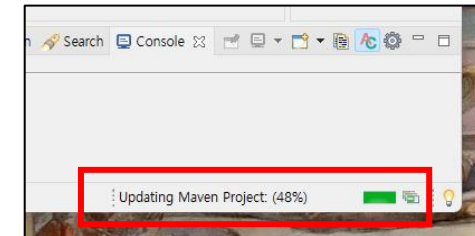
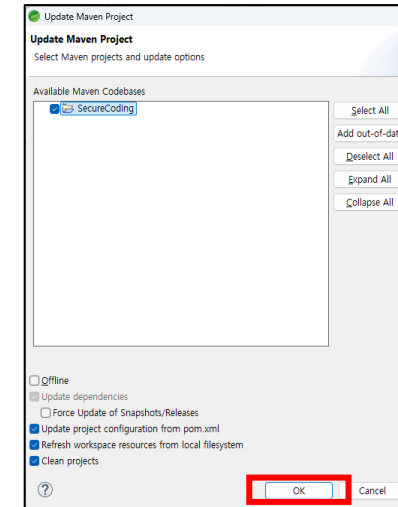
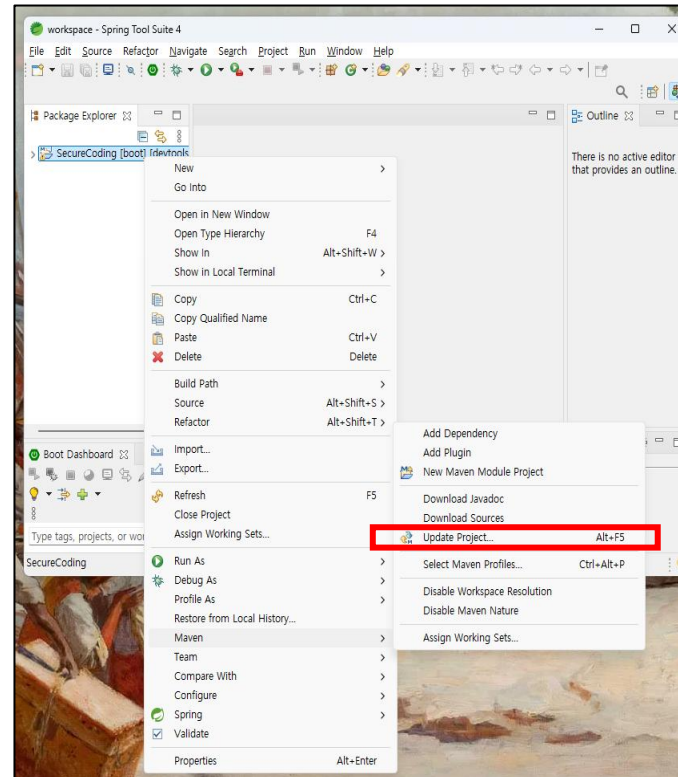
좌측 Project Explorer 영역에 SecureCoding 폴더가 보이면 정상



실습환경 구축 - 웹 서비스 서버 설정 (5/8)

Step5. Update Project 실행

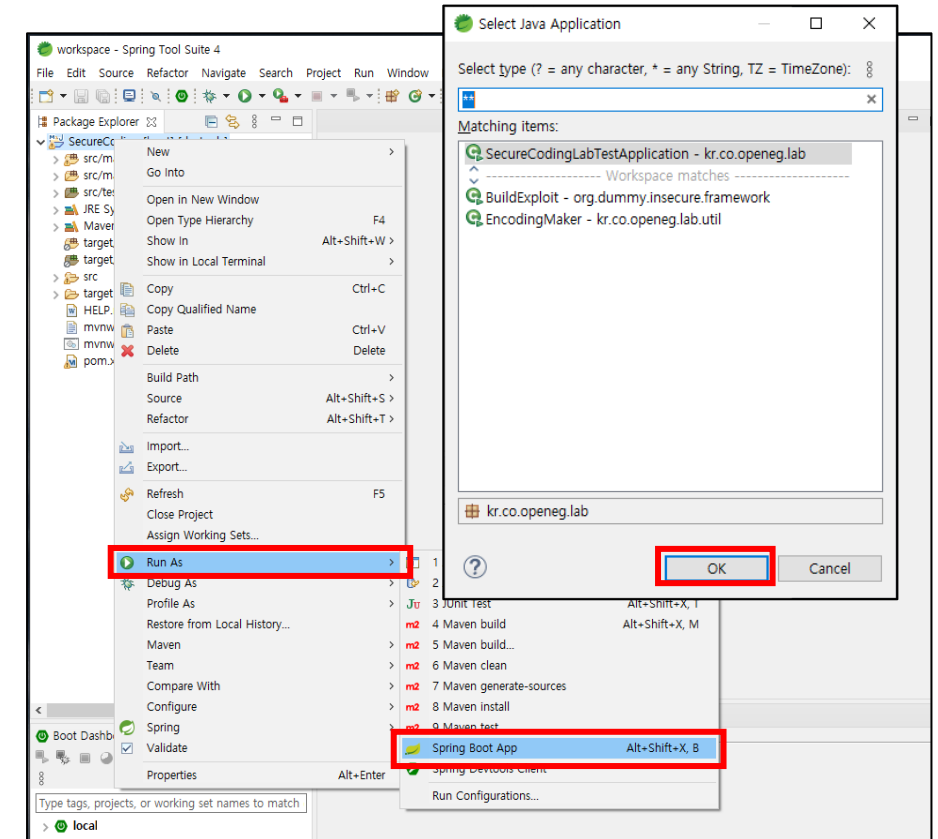
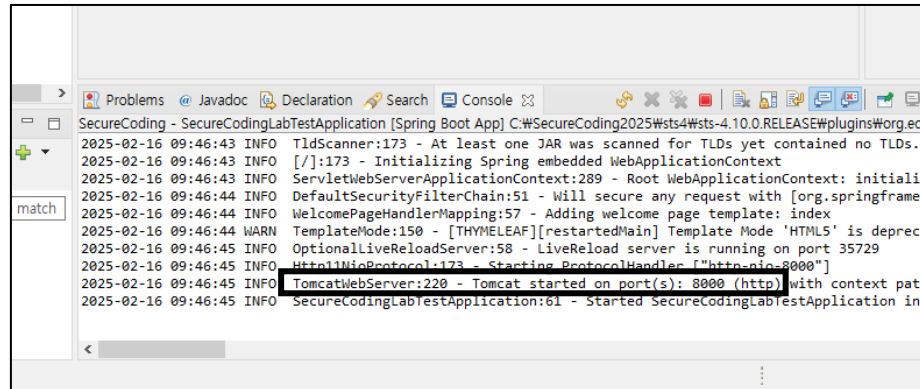
- SecureCoding 프로젝트의 의존성과 설정을 최신 상태로 동기화
프로젝트명에서 마우스 오른쪽 클릭 > Maven > Update Project 선택
Update Maven 창에서 OK 선택
- IDE의 우측 하단 진행 상황 확인
모든 업데이트가 완료되면 진행 상황이 사라짐



실습환경 구축 - 웹 서비스 서버 설정 (6/8)

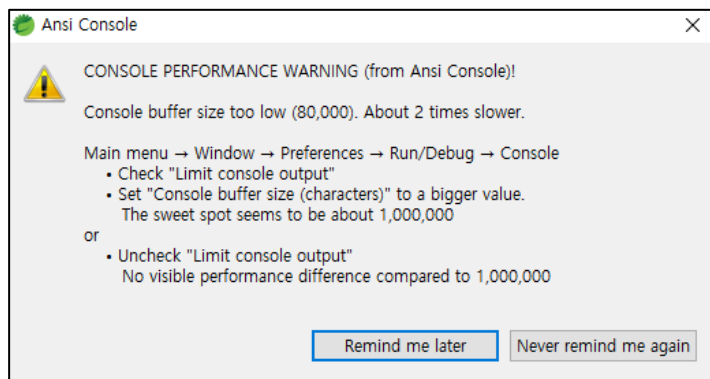
Step6. Run As > Spring Boot App 실행

- 프로젝트의 업데이트가 모두 완료되면 서버 실행
프로젝트명에서 마우스 오른쪽 클릭 > Run As > Spring boot App 선택
Select Java Application에서 SecureCodingLabTestApplication 확인 후 OK 선택
- IDE의 우측 하단 진행 상황 확인
아래와 같이 콘솔창에 Tomcat started on port(s): 8000 이 보이면 정상



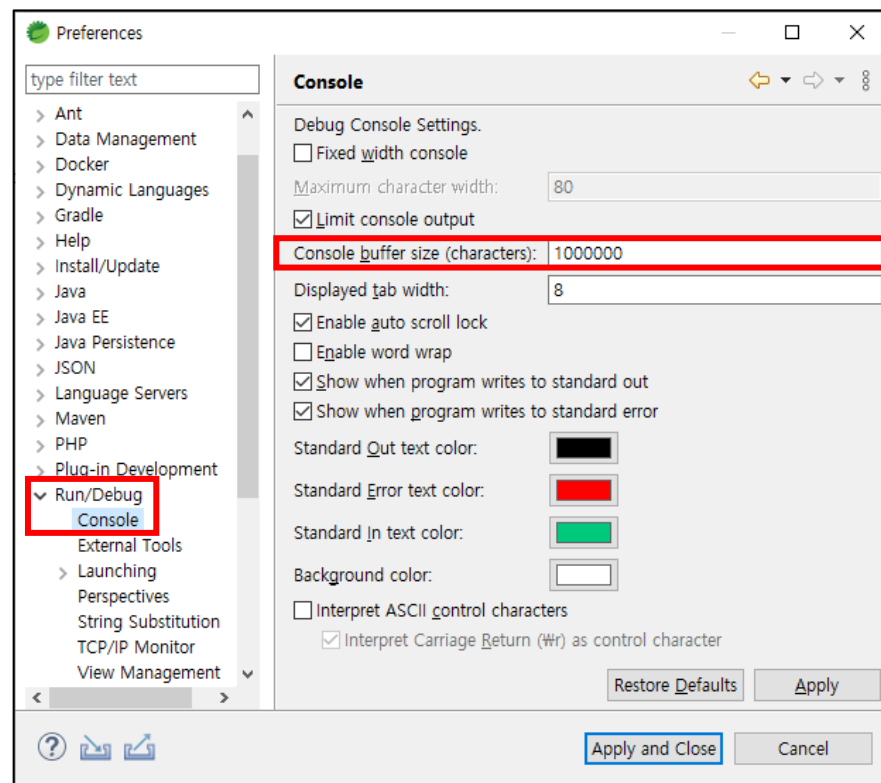
실습환경 구축 - 웹 서비스 서버 설정 (7/8)

Step7. 다음과 같은 경고가 발생하였을 때 대처 방법



- 콘솔 윈도우의 버퍼 크기를 늘려줘야 함

Window > Preferences > Run/Debug > Console을 선택하여 Console buffer size(characters)를 1,000,000으로 수정한 후, 'Apply and Close' 버튼 클릭

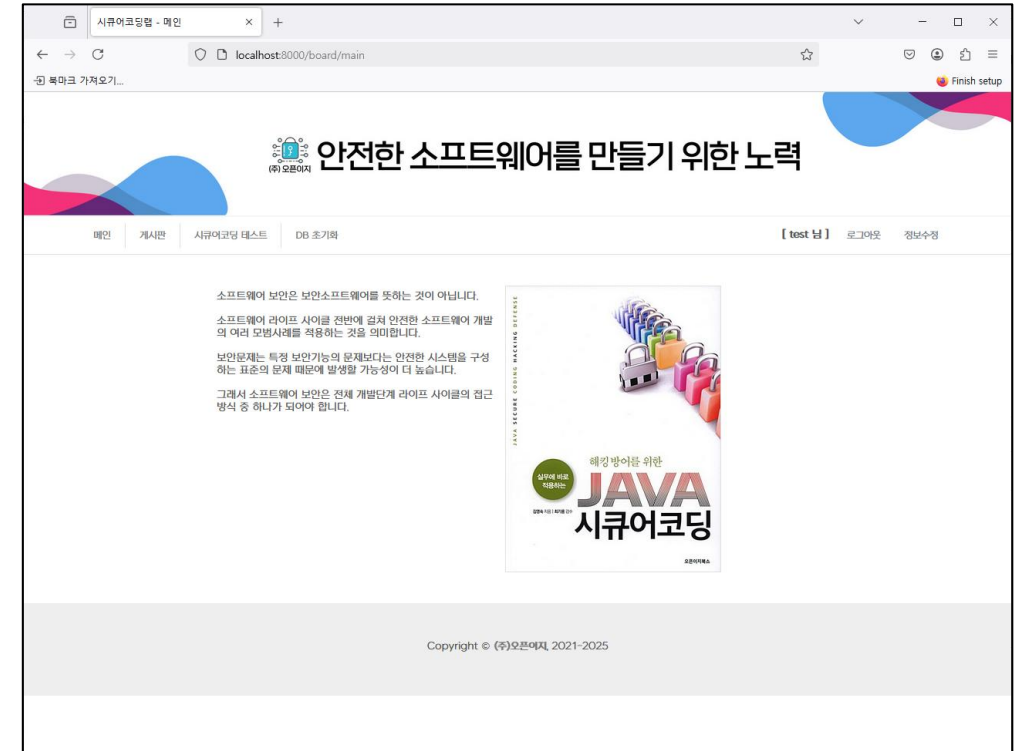
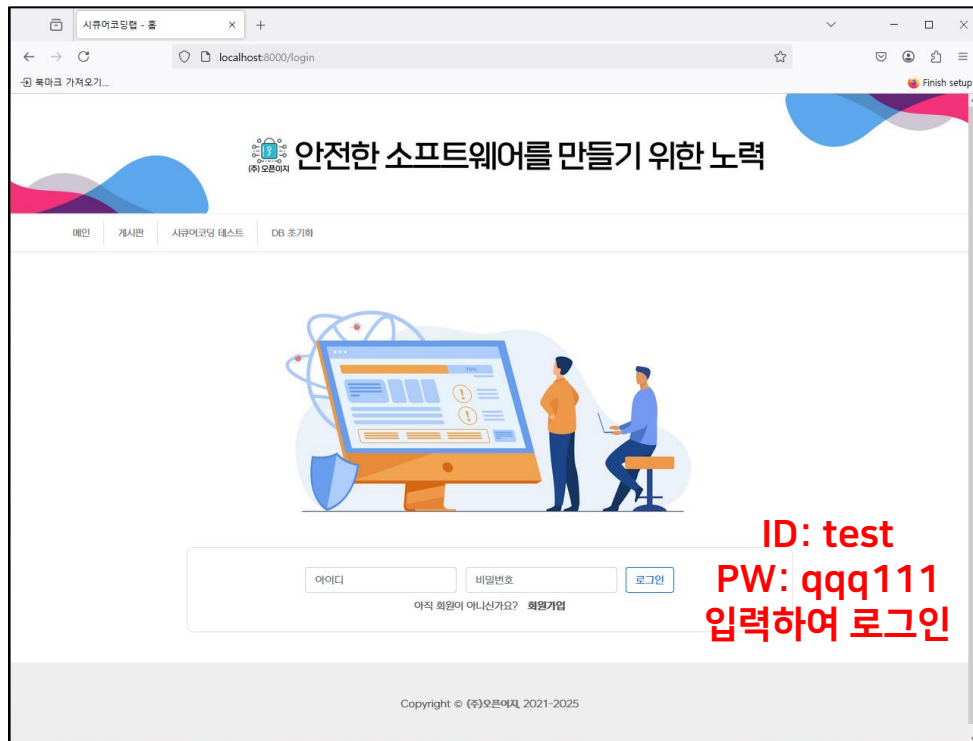


실습환경 구축 - 웹 서비스 서버 설정 (8/8)

Step8. 웹 브라우저를 띄워 정상 동작 확인

크롬이나 파이어폭스를 실행시켜 LAB 웹 서비스에 접속

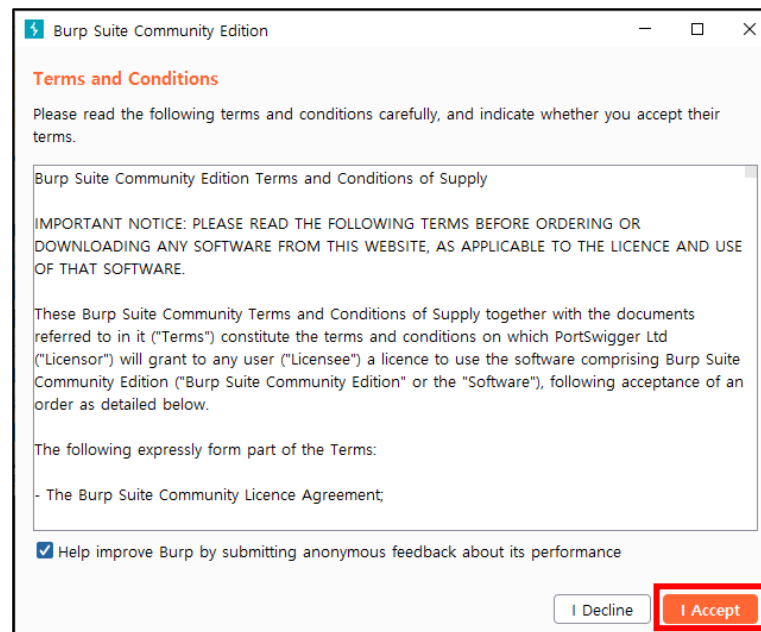
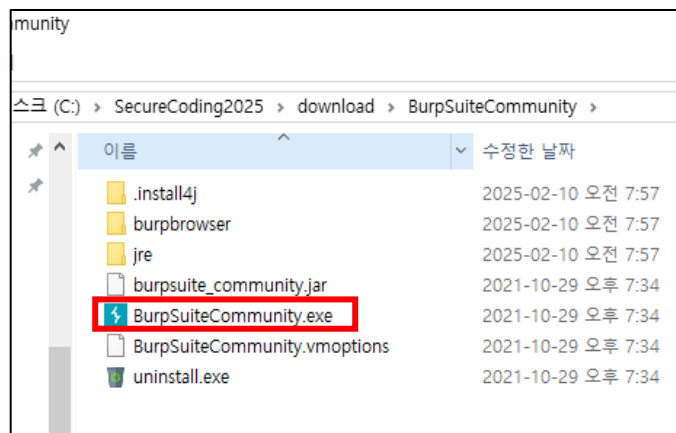
주소창에 localhost:8000 를 입력하면 아래와 같은 화면이 나타나고, 로그인 가능함



실습환경 구축 - 프록시 설정(1/4)

Step1. BurpSuite 패키지 실행

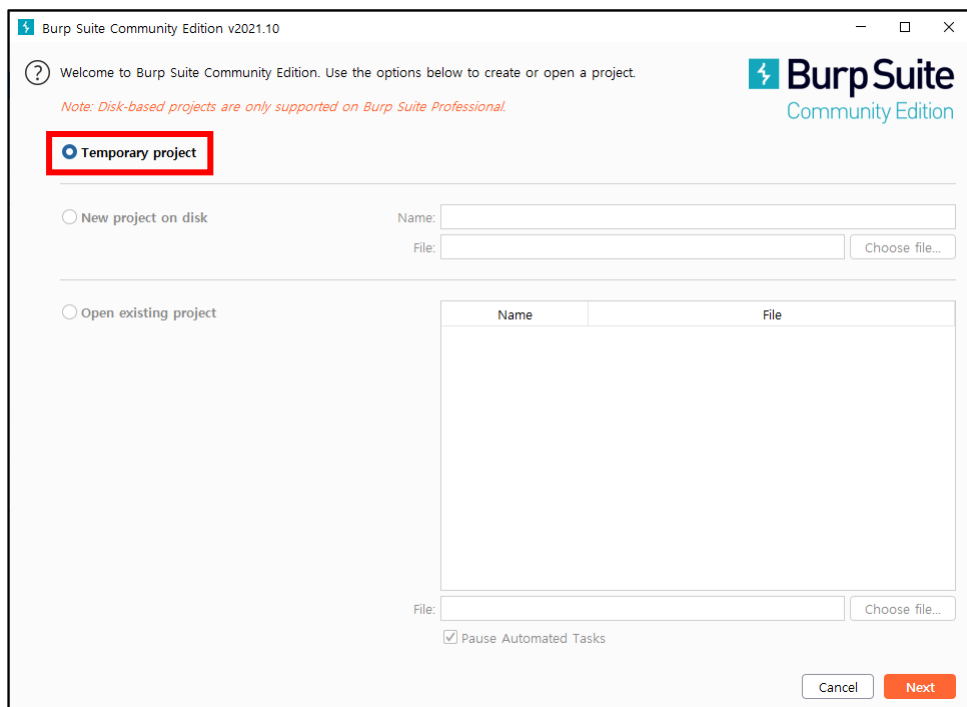
- BurpSuiteCommunity.exe 를 실행하면 별도 설치 없이 바로 구동됨
C:\SecureCoding2025\download\BurpSuiteCommunity 폴더에 있음
- Terms and Conditions 에서 'I Accept' 버튼 선택



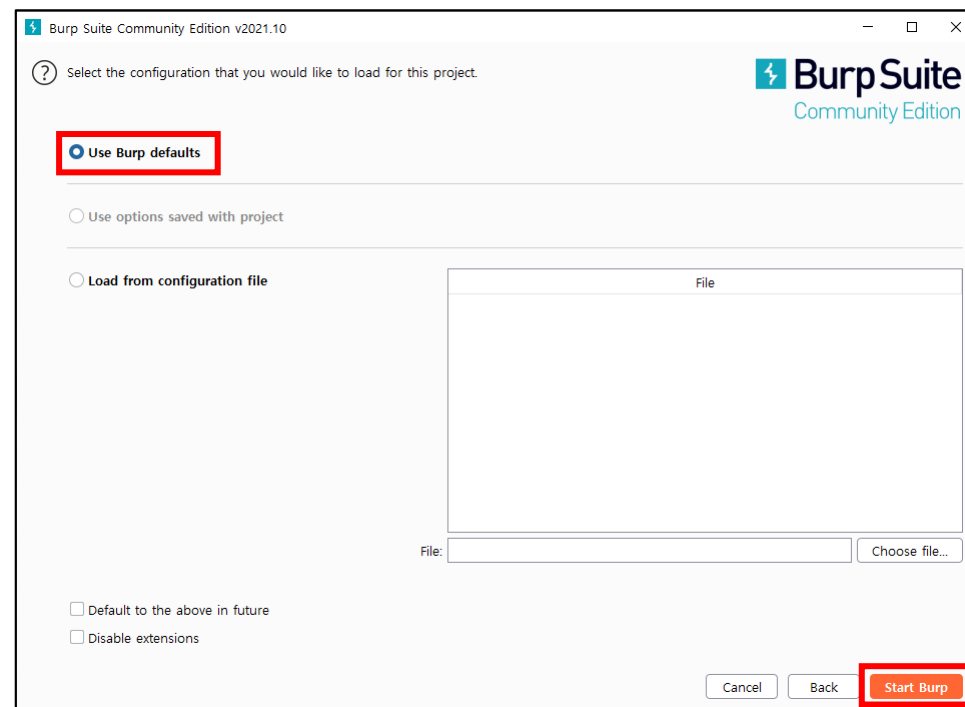
실습환경 구축 - 프록시 설정(2/4)

Step2. BurpSuite 패키지 실행 옵션 선택

Temporary project 선택 후 'Next' 버튼 선택



Use Burp defaults 선택 후 'Start Burp' 버튼 선택



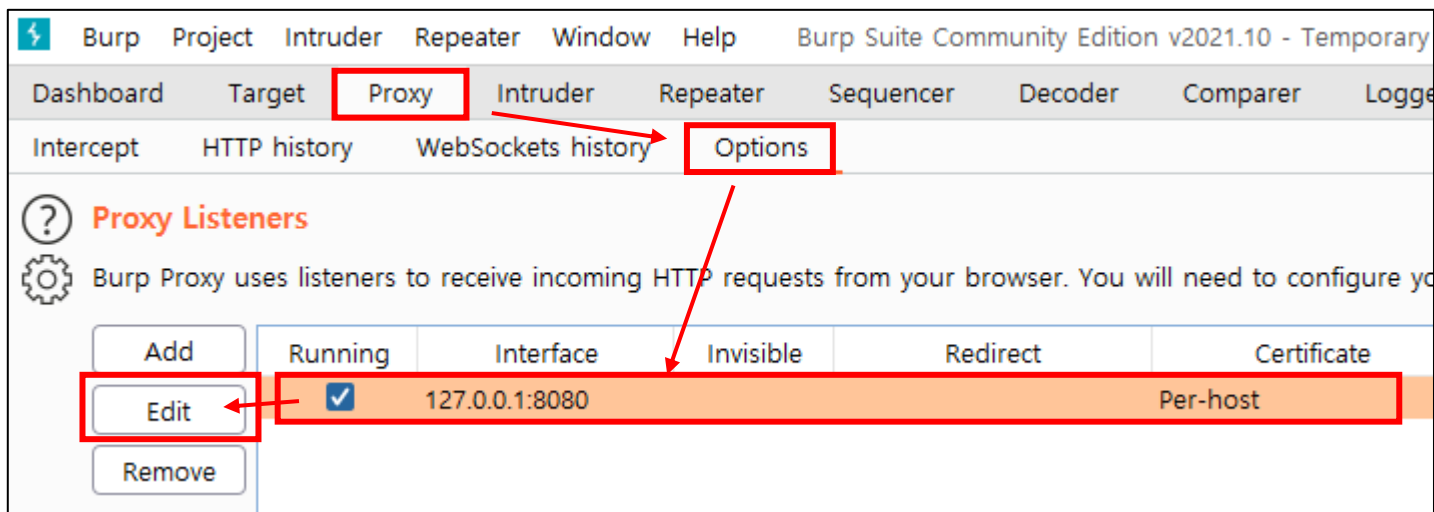
실습환경 구축 - 프록시 설정(3/4)

Step3. 프록시 서버의 인터페이스 설정 수정

- 상단 메뉴에서 Proxy > Options 선택

Proxy Listener 목록에서 127.0.0.1:8080 인터페이스 체크 및 선택
해당 인터페이스가 주황색으로 표시됨

- 좌측 Edit 버튼 클릭

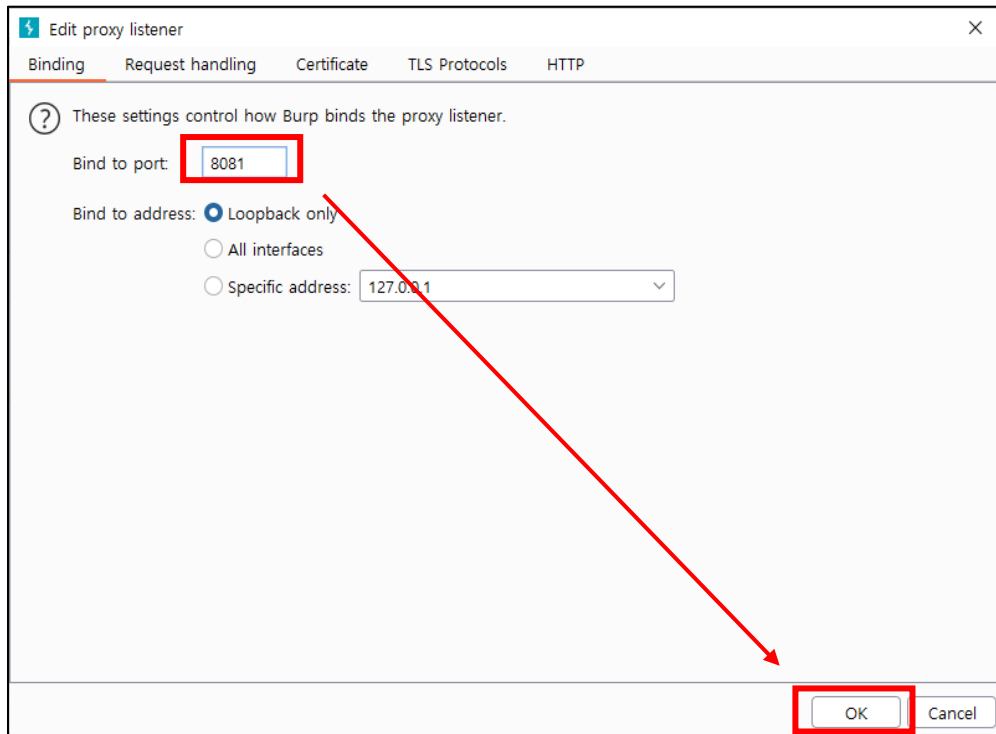


실습환경 구축 - 프록시 설정(4/4)

Step4. 프록시 서버의 Bind Port 수정

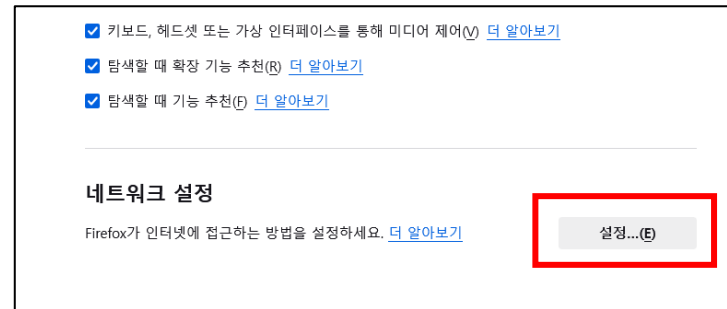
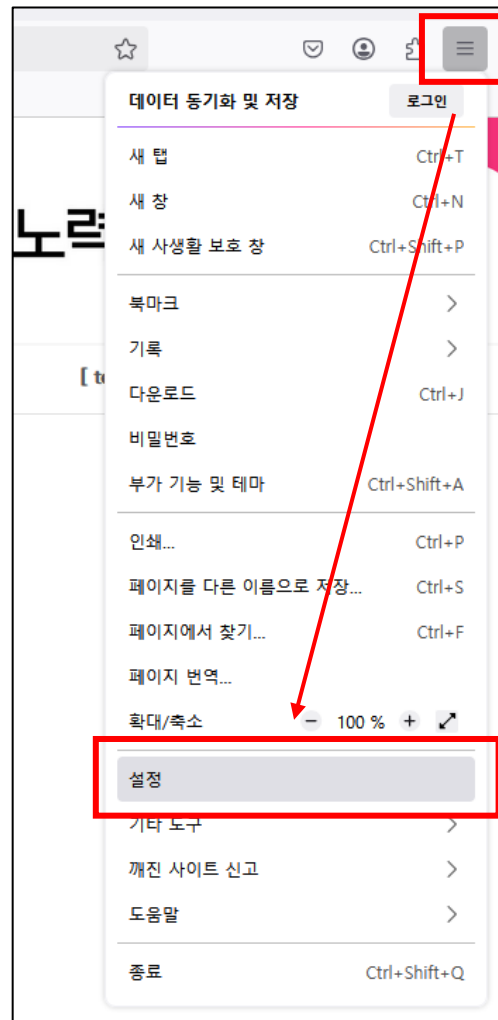
Edit proxy listener 창에서 'Bind to port' 항목을 8081로 변경

OK 버튼 선택



실습환경 구축 - 파이어폭스 브라우저 설정(1/3)

Step1. 우측 상단 세줄 메뉴 버튼 클릭
아래 항목에서 '설정' 항목 선택
나타난 '일반' 화면 맨 아래로 내려가서
'네트워크 설정'의 '설정' 버튼 선택



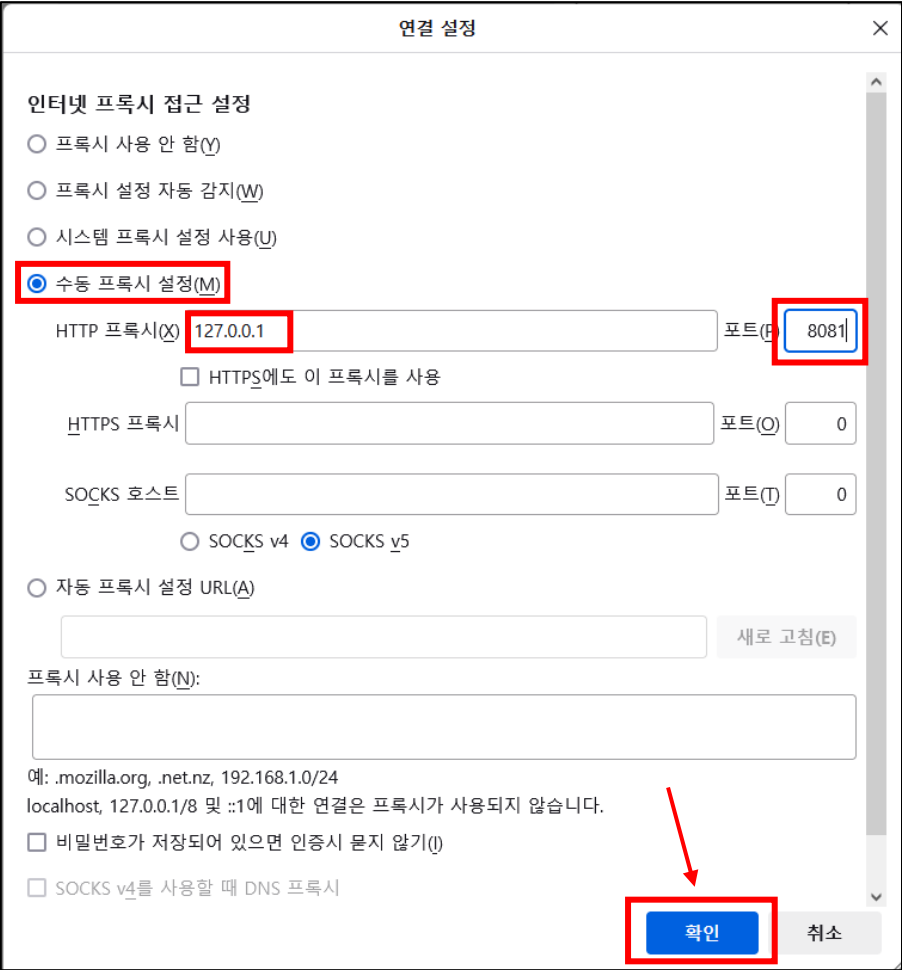
실습환경 구축 - 파이어폭스 브라우저 설정(2/3)

Step2. 연결 설정에서 수동 프록시 서버 정보 입력

수동 프록시 설정 선택

HTTP 프록시에 127.0.0.1, 포트에 8081 입력

확인 버튼 선택



The image shows the '연결 설정' (Connection Settings) dialog box in Firefox. The '인터넷 프록시 접근 설정' (Internet Proxy Access Settings) section is active. The '수동 프록시 설정(M)' (Manual Proxy Settings) radio button is selected and highlighted with a red box. Below it, the 'HTTP 프록시(X)' (HTTP Proxy) field is set to '127.0.0.1' and the '포트(P)' (Port) field is set to '8081', both highlighted with red boxes. The 'HTTPS 프록시' (HTTPS Proxy) and 'SOCKS 호스트' (SOCKS Host) fields are empty. The 'SOCKS v4' and 'SOCKS v5' radio buttons are both unselected. The '자동 프록시 설정 URL(A)' (Automatic Proxy Settings URL) field is empty. The '프록시 사용 안 함(N):' (No Proxy) section is empty. The '확인' (OK) button at the bottom right is highlighted with a red box and has a red arrow pointing to it. The '취소' (Cancel) button is also visible.

인터넷 프록시 접근 설정

- ☐ 프록시 사용 안 함(Y)
- ☐ 프록시 설정 자동 감지(W)
- ☐ 시스템 프록시 설정 사용(U)
- ☒ 수동 프록시 설정(M)

HTTP 프록시(X) 127.0.0.1 포트(P) 8081

☐ HTTPS에도 이 프록시를 사용

HTTPS 프록시 포트(O) 0

SOCKS 호스트 포트(I) 0

☐ SOCKS v4 ☒ SOCKS v5

☐ 자동 프록시 설정 URL(A)

새로 고침(E)

프록시 사용 안 함(N):

예: .mozilla.org, .net.nz, 192.168.1.0/24
localhost, 127.0.0.1/8 및 ::1에 대한 연결은 프록시가 사용되지 않습니다.

☐ 비밀번호가 저장되어 있으면 인증시 묻지 않기(I)

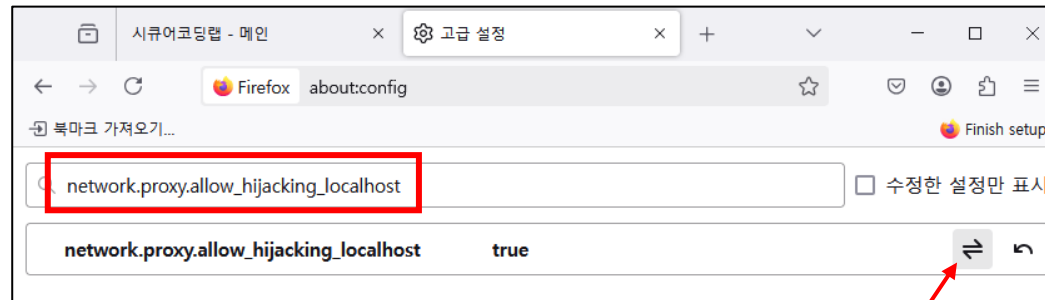
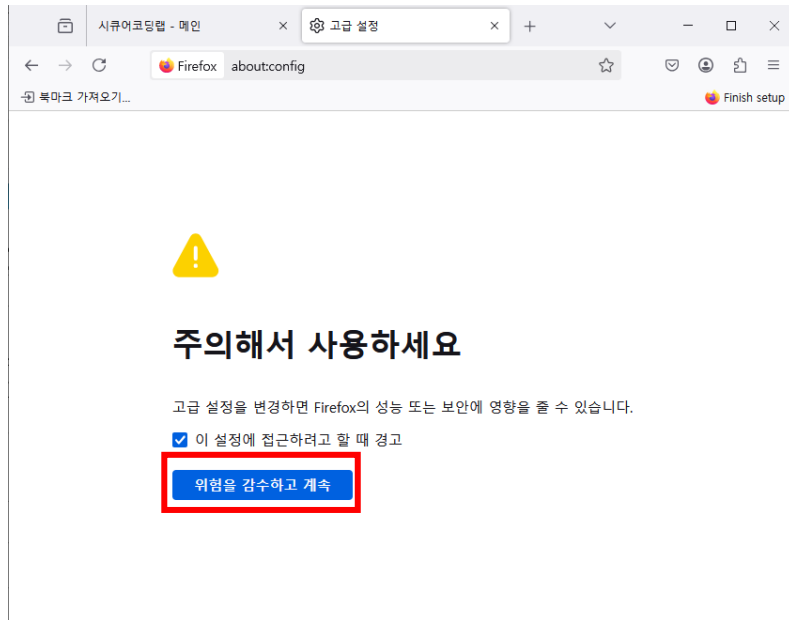
☐ SOCKS v4를 사용할 때 DNS 프록시

확인 취소

실습환경 구축 - 파이어폭스 브라우저 설정(3/3)

Step3. localhost 에서 프록시 설정 가능하도록 옵션 변경

- 주소창에 'about:config' 입력
- 경고 메시지에서 '위험을 감수하고 계속' 버튼 선택
- '설정 이름 검색' 창에 'network.proxy.allow_hijacking_localhost'를 입력
- 해당 설정 우측 좌우 화살표 버튼을 눌러 값을 'true'로 변경



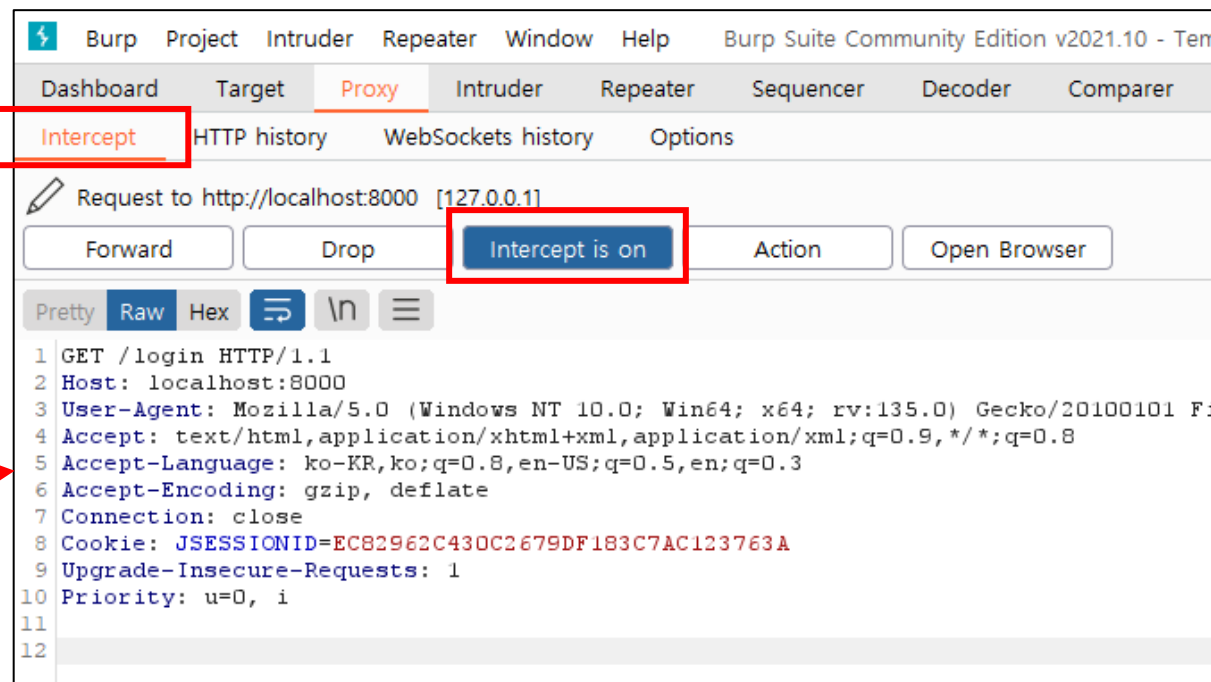
실습환경 구축 - BurpSuite 프록시 서버 동작

이후 해당 웹 브라우저 사용시 발생하는 모든 HTTP 요청은 BurpSuite가 가로 챔

웹 브라우저에서 LAB 웹 서비스 주소를 입력해도 일단 BurpSuite가 붙잡고 있어서 화면이 나오지 않음

Intercept 메뉴에서 'Intercept is on' 버튼을 눌러 off 시키면, 대기하던 모든 내용이 전달되어 화면이 나타남

불잡혀있는
HTTP 요청





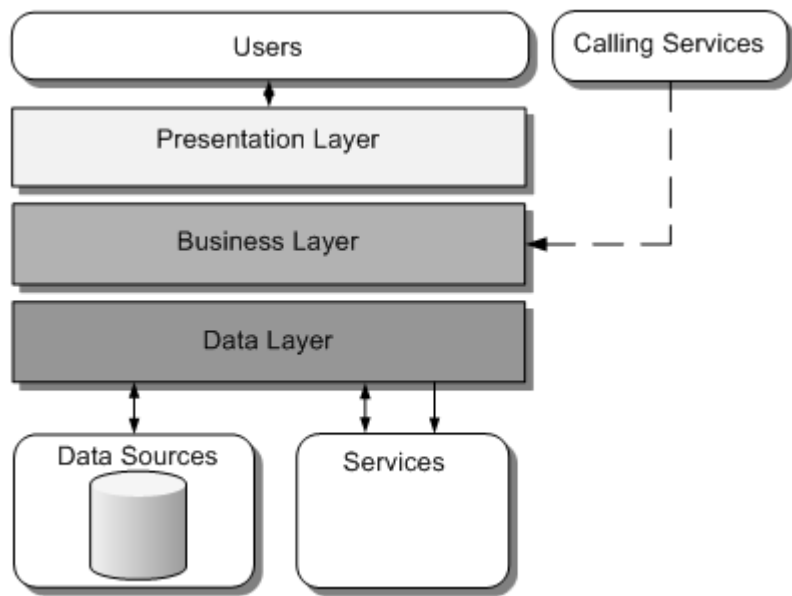
실습환경 구축-1

01 실습환경 구축

02 웹과 HTTP 구조 이해

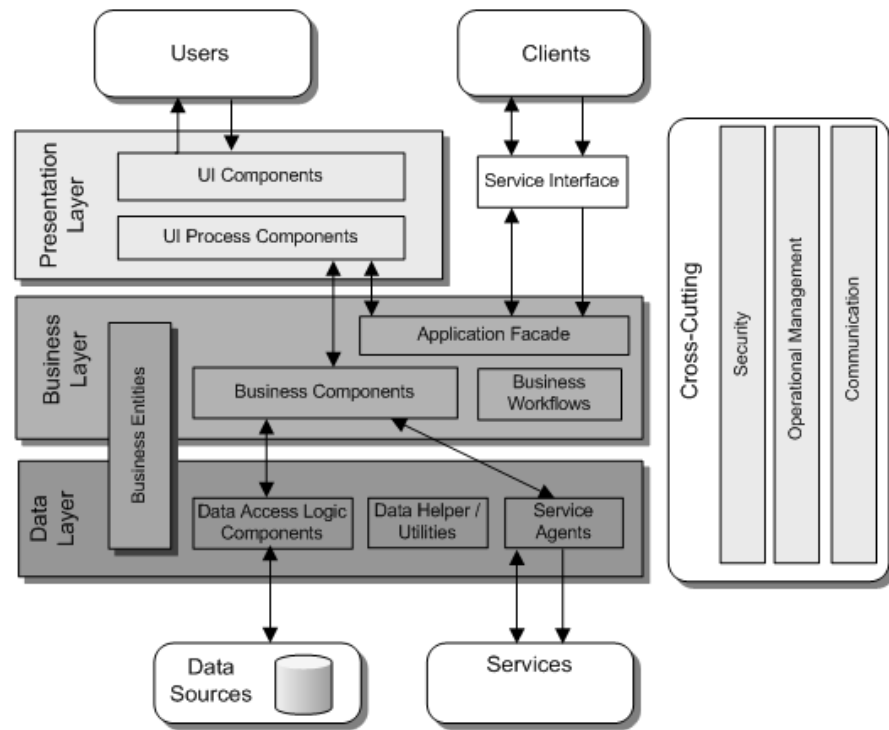
03 데이터 인코딩

웹 구조



단순화된 구조

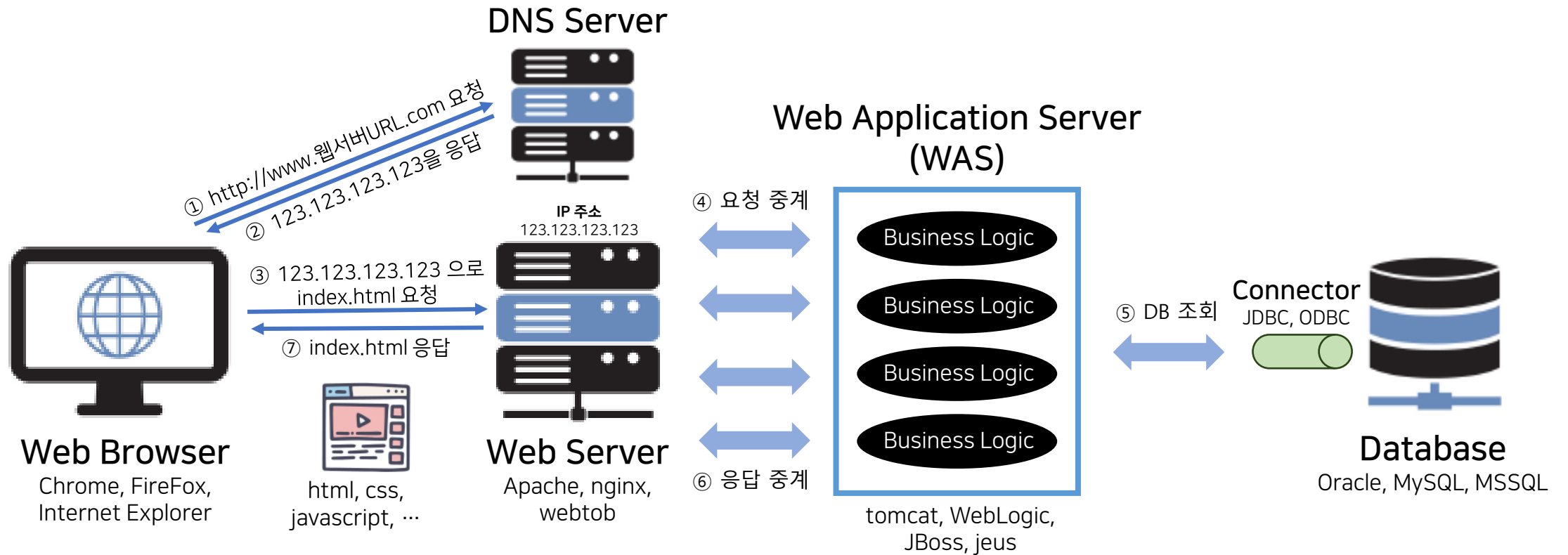
출처: 위키피디아



확장 표현된 구조

출처: <https://medium.com/@iyuroch/application-architecture-plan-layers-68defde66ade>

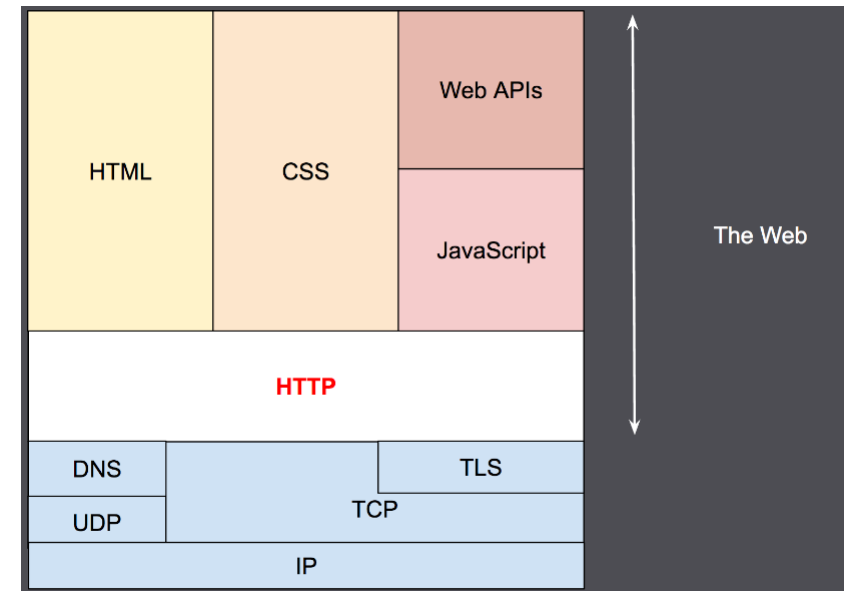
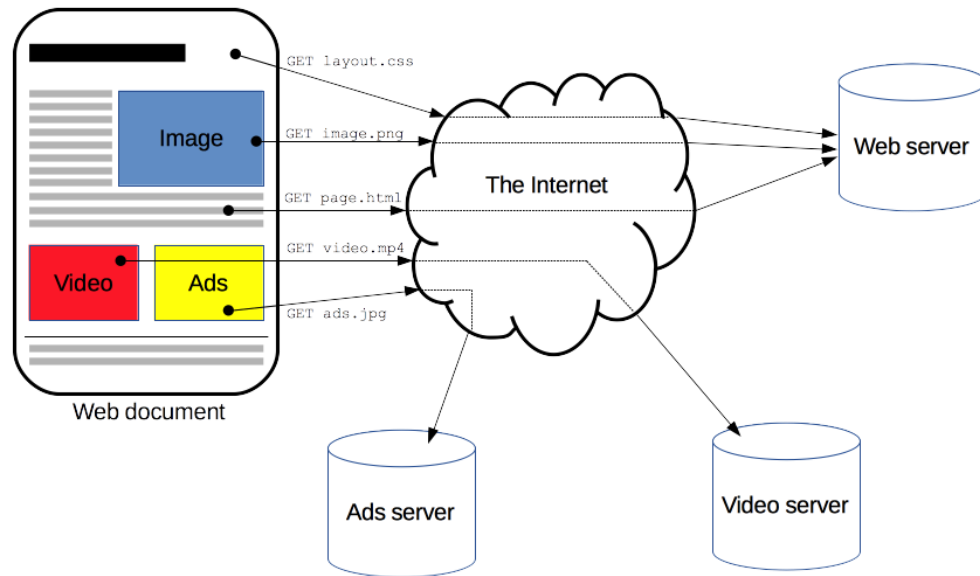
웹 서비스 흐름



HTTP 구조

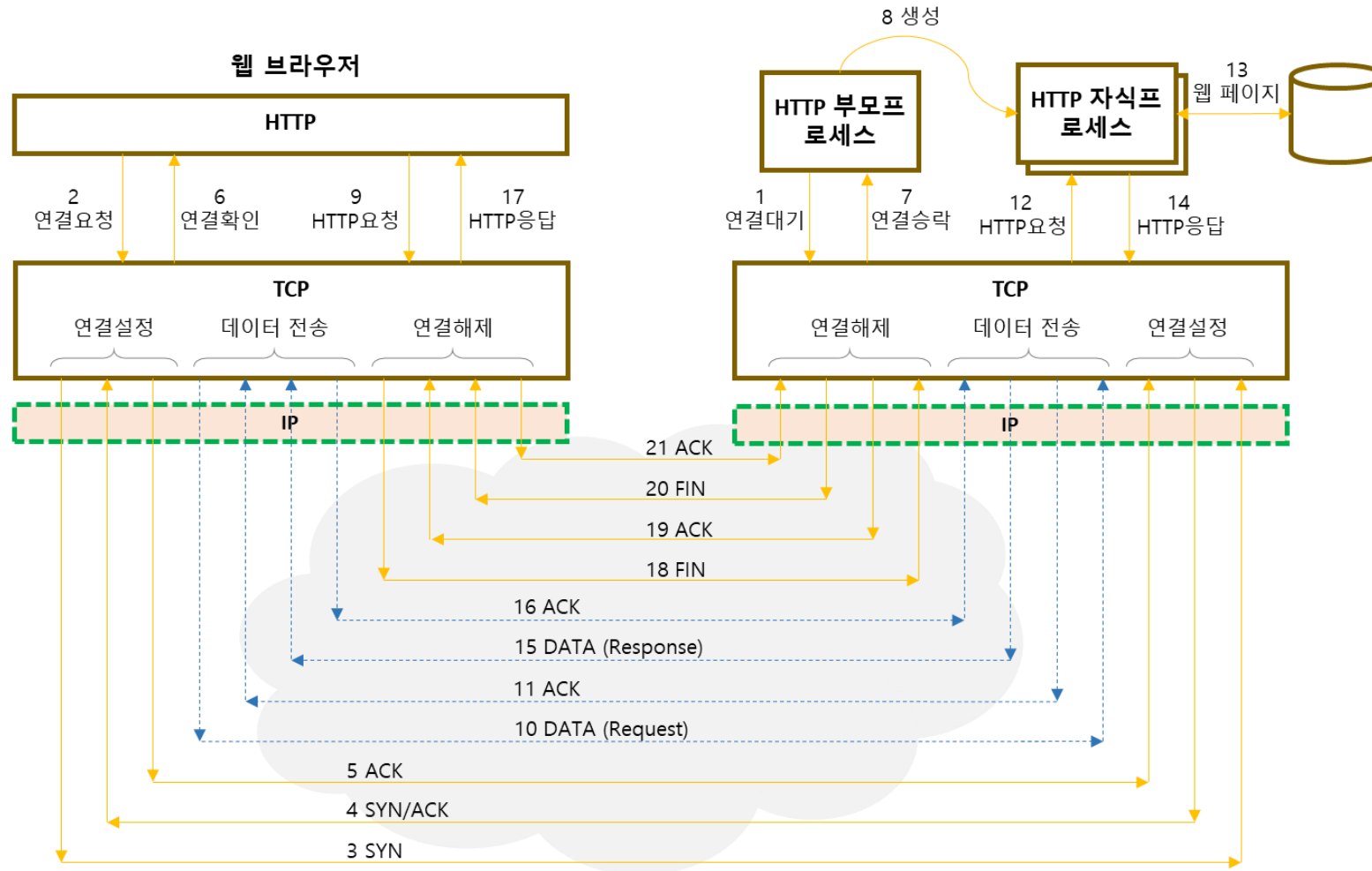
HTTP (HyperText Transfer Protocol)

- WWW 상에서 HTML 문서와 같은 리소스들을 가져올 수 있도록 해주는 프로토콜
- 요청(Request), 응답(Response) 구조, 무상태 방식(Stateless)



HTTP 구조

HTTP 흐름 이해



BurpSuite을 활용한 HTTP 구조 보기

- 수동 웹 프록시로 BurpSuite를 경유하도록 설정한 경우

Proxy > HTTP history 에서 HTTP
Request/Response 내용 조회 가능

The screenshot displays the Burp Suite interface. The top menu bar includes 'Burp', 'Project', 'Intruder', 'Repeater', 'Window', and 'Help'. Below the menu, the 'Proxy' tab is selected, and the 'HTTP history' sub-tab is active. A table lists intercepted HTTP requests. The first request is highlighted, showing details like Host (localhost:8000), Method (GET), and URL (/board/main). Below the table, the 'Request' and 'Response' sections are expanded, showing the raw HTTP data. The 'Request' section shows a GET request for /board/main. The 'Response' section shows an HTTP 200 response with various headers and an HTML body. The 'INSPECTOR' panel on the right shows the structure of the request and response, including attributes, cookies, headers, and body.

#	Host	Method	URL	Params	Edited	Status	Length	MIME type	Extension	Title
197	http://localhost:8000	GET	/board/main			200	3589	HTML		...
202	http://detectportal.firefox.com	GET	/canonical.html			200	317	XML	html	
203	http://detectportal.firefox.com	GET	/success.txt?ipv4		✓	200	235	text	txt	
204	http://detectportal.firefox.com	GET	/success.txt?ipv6		✓	200	235	text	txt	

Request

```
1 GET /board/main HTTP/1.1
2 Host: localhost:8000
3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:135.0) Gecko/20100101 Firefox/135.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: ko-KR,ko;q=0.8,en-US;q=0.5,en;q=0.3
6 Accept-Encoding: gzip, deflate
7 Referer: http://localhost:8000/login
8 Connection: close
9 Cookie: JSESSIONID=C84105A85AC5C292FEE00982DB9EA846
10 Upgrade-Insecure-Requests: 1
11 Priority: u=0, i
12
13
```

Response

```
1 HTTP/1.1 200
2 Vary: Origin
3 Vary: Access-Control-Request-Method
4 Vary: Access-Control-Request-Headers
5 X-Content-Type-Options: nosniff
6 X-XSS-Protection: 1; mode=block
7 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
8 Pragma: no-cache
9 Expires: 0
10 Content-Type: text/html; charset=UTF-8
11 Content-Language: ko-KR
12 Date: Sun, 16 Feb 2025 05:15:10 GMT
13 Connection: close
14 Content-Length: 3182
15
16 <!DOCTYPE html>
17 <html xmlns="http://www.w3.org/1999/xhtml">
18
19 <head>
20 <meta charset="utf-8" />
21 <meta name="viewport" content="width=device-width, initial-scale=1,

```

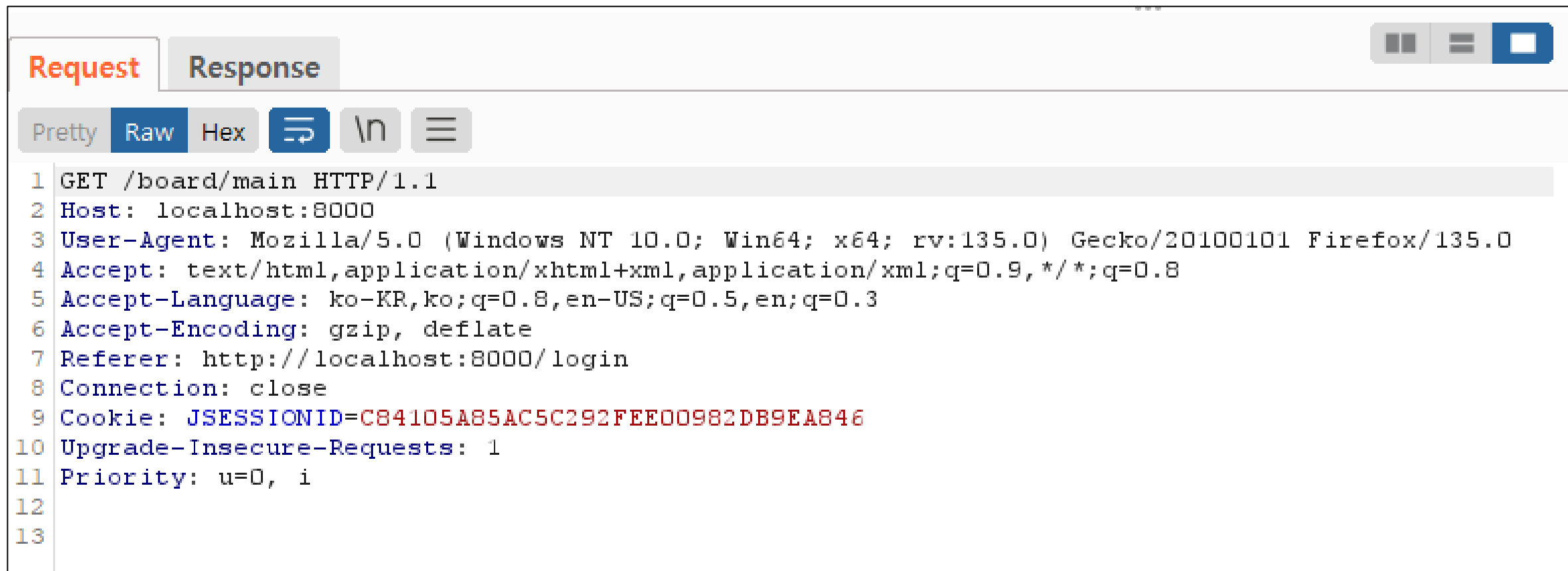
INSPECTOR

- Request Attributes
- Request Cookies (1)
- Request Headers (10)
- Response Headers (13)

Request
내용

Response
내용

HTTP Request 메시지 구조



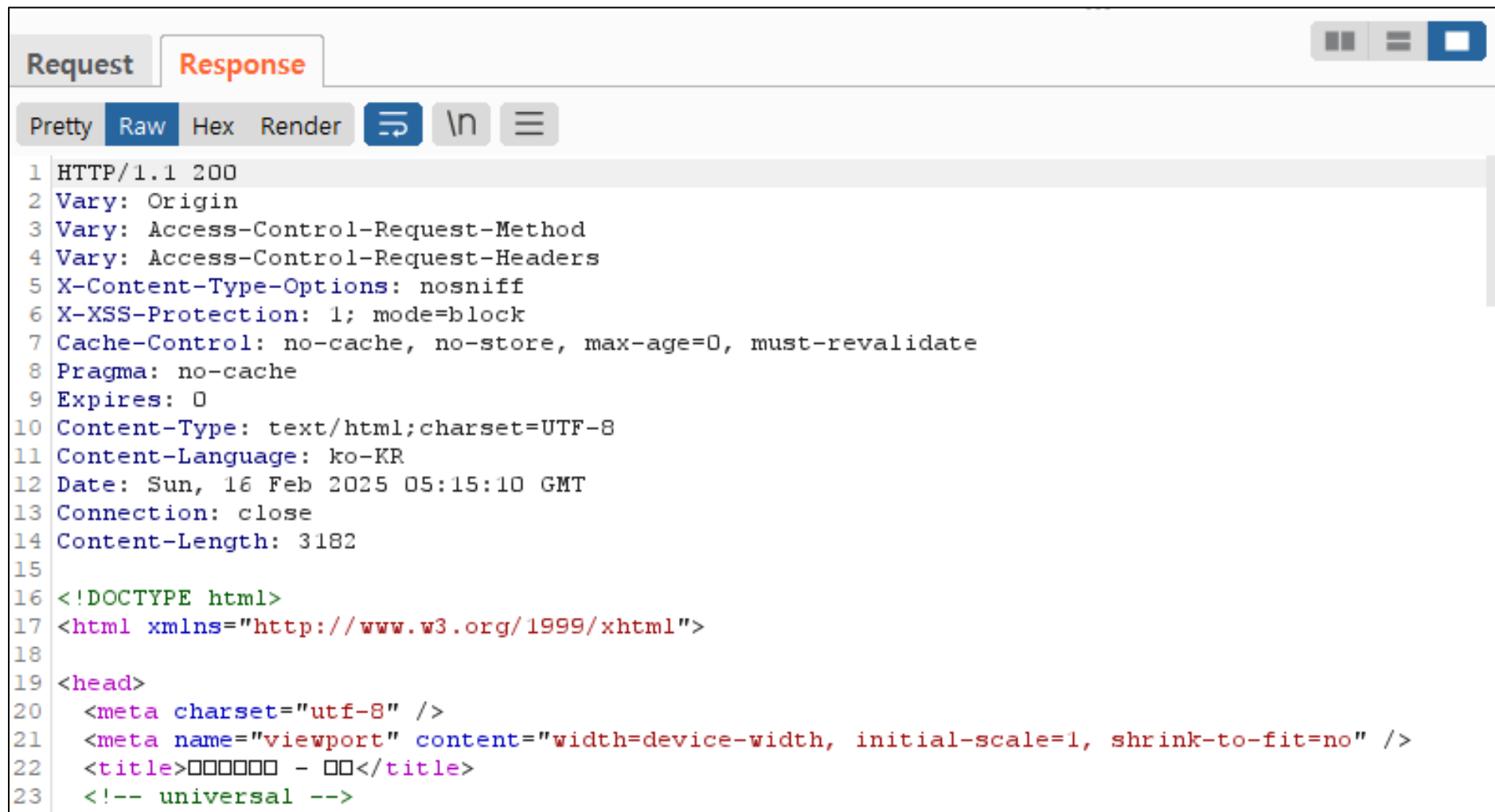
The screenshot displays the 'Request' tab of a web browser's developer tools. The 'Response' tab is also visible. The request is a GET to /board/main. The headers are as follows:

- 1 GET /board/main HTTP/1.1
- 2 Host: localhost:8000
- 3 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64; rv:135.0) Gecko/20100101 Firefox/135.0
- 4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
- 5 Accept-Language: ko-KR,ko;q=0.8,en-US;q=0.5,en;q=0.3
- 6 Accept-Encoding: gzip, deflate
- 7 Referer: http://localhost:8000/login
- 8 Connection: close
- 9 Cookie: JSESSIONID=C84105A85AC5C292FEE00982DB9EA846
- 10 Upgrade-Insecure-Requests: 1
- 11 Priority: u=0, i
- 12
- 13

HTTP Request Method

Method	설 명
GET	자원 요청(default값), 요청 파라미터를 URL에 붙여서 전송 보안이 요구되지 않은 짧은 데이터를 전송하는 경우, 북마크가 필요한 경우에 사용 test.jsp?id=kim&pass=1234
POST	자원 요청, 요청 파라미터를 Body에 붙여서 전송 전송할 데이터가 양이 많은 경우, 비밀 정보를 전송해야 하는 경우에 사용 <form method="post"></form>
HEAD	HTTP Header 정보만 요청 Meta 정보 획득이나 Hyper Text의 링크 유효성을 검사하는 용도로 사용
PUT	URL 자원을 생성하도록 요청. 파일을 업로드 할 수 있다.
DELETE	URL 자원을 삭제하도록 요청. 파일을 삭제할 수 있다.
TRACE	진단을 위한 목적으로 사용되는 메서드로, 서버는 응답 바디에 요청 받은 자료와 똑같은 내용을 보냄
OPTIONS	응답 가능한 HTTP Method 를 요청

HTTP Response 메시지 구조



```
1 HTTP/1.1 200
2 Vary: Origin
3 Vary: Access-Control-Request-Method
4 Vary: Access-Control-Request-Headers
5 X-Content-Type-Options: nosniff
6 X-XSS-Protection: 1; mode=block
7 Cache-Control: no-cache, no-store, max-age=0, must-revalidate
8 Pragma: no-cache
9 Expires: 0
10 Content-Type: text/html; charset=UTF-8
11 Content-Language: ko-KR
12 Date: Sun, 16 Feb 2025 05:15:10 GMT
13 Connection: close
14 Content-Length: 3182
15
16 <!DOCTYPE html>
17 <html xmlns="http://www.w3.org/1999/xhtml">
18
19 <head>
20   <meta charset="utf-8" />
21   <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no" />
22   <title>□□□□□□ - □□</title>
23   <!-- universal -->
```

HTTP Response Status Code

상태코드	설명
1xx	정보를 제공함
2xx	요청이 성공적으로 이루어짐
3xx	요청한 해당 자원이 다른 곳에 있음
4xx	요청에 문제가 있음
5xx	서버에 에러가 있음

100(계속)	요청자는 요청을 계속해야 한다. 서버는 이 코드를 제공하여 요청의 첫 번째 부분을 받았으며 나머지를 기다리고 있음
101(프로토콜 전환)	요청자가 서버에 프로토콜 전환을 요청했으며 서버는 이를 승인하는 중.
200(성공)	서버가 요청을 정상적으로 처리함.
201(작성됨)	성공적으로 요청되었으며 서버가 새 리소스를 작성
202(허용됨)	서버가 요청을 접수했지만 아직 처리하지 않음
203(신뢰할 수 없는 정보)	서버가 요청을 성공적으로 처리했지만 다른 소스에서 수신된 정보를 제공
204(콘텐츠 없음)	서버가 요청을 성공적으로 처리했지만 콘텐츠를 제공하지 않음
205(콘텐츠 재설정)	서버가 요청을 성공적으로 처리했지만 콘텐츠를 표시하지 않음
206(일부 콘텐츠)	204 응답과 달리 이 응답은 요청자가 문서 보기를 재설정할 것을 요구(예: 새 입력을 위한 양식 비우기). 서버가 GET 요청의 일부만 성공적으로 처리함

HTTP Response Status Code

300(복수 응답)	서버가 요청에 따라 여러 조치를 선택. 수행할 작업을 선택하거나, 요청자가 선택할 수 있는 작업 목록을 제공.
301(영구 이동)	요청한 페이지가 새 위치로 영구적으로 이동했음.
302(임시 이동)	현재 서버가 다른 위치의 페이지로 요청에 응답하고 있지만, 요청자는 향후 요청 시 원래 위치를 계속 사용해야 함.
303(기타 위치 보기)	HEAD 요청 이외의 모든 요청을 다른 위치로 자동으로 전달
304(수정되지 않음)	마지막 요청 이후 요청한 페이지는 수정되지 않았음.
305(프록시 사용)	요청자는 프록시를 사용하여 요청한 페이지만 접근할 수 있음.
307(임시 리다이렉션)	현재 서버가 다른 위치의 페이지로 요청에 응답하고 있지만, 요청자는 향후 요청시 원래 위치를 계속 사용해야 함.
308(영구 리다이렉션)	RFC에서 실험적으로 승인됨
400(잘못된 요청)	서버가 요청의 구문을 인식하지 못함
401(권한 없음)	이 요청은 인증이 필요함. 서버는 로그인에 필요한 페이지에 대해 이 요청을 제공할 수 있음.
402(결제 필요)	이 요청은 결제가 필요함
403(Forbidden, 금지됨)	서버가 요청을 거부함. 예를 들어, 사용자가 리소스에 대한 필요 권한을 갖고 있지 않다. (401은 인증 실패, 403은 인가 실패라고 볼 수 있음)
404(Not Found, 찾을 수 없음)	서버가 요청한 페이지(Resource)를 찾을 수 없음
405(허용되지 않는 방법)	요청에 지정된 방법을 사용할 수 없음 ex)POST 방식으로 요청을 받는 서버에 GET 요청을 보내는 경우, 또는 읽기 전용 리소스에 PUT 요청을 보내는 경우
406(허용되지 않음)	요청한 페이지가 요청한 콘텐츠 특성으로 응답할 수 없음
407(프록시 인증 필요)	이 상태 코드는 401(권한 없음)과 비슷하지만 요청자가 프록시를 사용하여 인증해야 함.
408(요청 시간초과)	서버의 요청 대기가 시간을 초과함
409(충돌)	서버가 요청을 수행하는 중에 충돌이 발생함. 서버는 PUT 요청과 충돌하는 PUT 요청에 대한 응답으로 이 코드를 요청 간 차이점 목록과 함께 표시

HTTP Response Status Code

410(사라짐)

411(길이 필요)

412(사전조건 실패)

413(요청 속성이 너무 큼)

414(요청 URI가 너무 김)

415(지원되지 않는 미디어 유형)

416(처리할 수 없는 요청범위)

417(예상 실패)

요청한 리소스가 영구적으로 삭제되었을 때 이 응답을 표시

서버는 유효한 콘텐츠 길이 헤더 입력란 없이는 요청을 수락하지 않음

서버가 요청자가 요청 시 부과한 사전조건을 만족하지 않음

요청이 너무 커서 서버가 처리할 수 없음

요청 URI(일반적으로 URL)가 너무 길어 서버가 처리할 수 없음

요청이 요청한 페이지에서 지원하지 않는 형식으로 되어 있음

요청이 페이지에서 처리할 수 없는 범위에 해당되는 경우 서버는 이 상태 코드를 표시

서버는 Expect 요청 헤더 입력란의 요구사항을 만족할 수 없음

500(내부 서버 오류)

501(구현되지 않음)

502 (Bad Gateway, 불량 게이트웨이)

503(서비스를 사용할 수 없음)

504(게이트웨이 시간초과)

505(HTTP 버전이 지원되지 않음)

506(Variant Also Negotiates, RFC 2295)

507(용량 부족, WebDAV; RFC 4918)

508(루프 감지됨, WebDAV; RFC 5842)

509(대역폭 제한 초과, Apache bw/limited extension)

510(확장되지 않음, RFC 2774)

511(네트워크 인증 필요, RFC 6585)

서버에 오류가 발생하여 요청을 수행할 수 없음

서버에 요청을 수행할 수 있는 기능이 없음. ex)서버가 요청 메소드를 인식하지 못할 때 이 코드를 표시

서버가 게이트웨이나 프록시 역할을 하고 있거나 또는 업스트림 서버에서 잘못된 응답을 받음

서버가 오버로드되었거나 유지관리를 위해 다운되었기 때문에 현재 서버를 사용할 수 없음

서버가 게이트웨이나 프록시 역할을 하고 있거나 또는 업스트림 서버에서 제때 요청을 받지 못함

서버가 요청에 사용된 HTTP 프로토콜 버전을 지원하지 않음



실습환경 구축-1

01 실습환경 구축

02 웹과 HTTP 구조 이해

03 데이터 인코딩

데이터 인코딩 개요

데이터를 특정 규칙에 따라 다른 형식으로 변환하는 과정

인코딩이 필요한 이유

- 데이터 전송: 다양한 환경에서 데이터가 올바르게 해석되도록 보장
- 보안 강화: 입력 값을 안전하게 처리하여 XSS, HTML 인젝션 등 보안 취약점 예방
- 호환성: 서로 다른 시스템 간에 데이터 포맷의 차이로 인한 오류 방지

인코딩 vs 암호화

- 인코딩은 데이터를 다른 형태로 변환해도, 누구나 역 변환(디코딩) 가능
- 암호화는 보안 목적으로 데이터를 변환하며, 복호화를 위해 키가 필요

주요 인코딩 유형

- BASE64 인코딩
- URL 인코딩

BASE64 인코딩

이진 데이터를 텍스트 형식으로 변환하는 인코딩 방식

특징

- 데이터를 6비트 단위로 분할하여, 총 64개의 문자 집합(A-Z, a-z, 0-9, +, /)으로 표현
- 6비트씩 처리하므로 부족한 비트는 0으로 패딩 처리되며, =로 표현

BASE64를 사용하는 이유

- 이메일, JSON, XML 등 텍스트 전송 환경에서 안전하게 사용 가능
- 네트워크나 시스템간 전송 시 발생할 수 있는 데이터 손상 및 인코딩 문제를 방지

인코딩 사례

Hello, World!



SGVsbG8sIFdvcmxklQ==

URL 인코딩

URL에서 사용이 불가능한 문자를 안전하게 표현하기 위해, 특정 문자를 % 기호와 ASCII 16진수 코드로 변환하는 방식

- URL 에서 사용할 수 없는 문자 처리

URL에서는 공백, ?, &, #, /, : 등의 문자들이 특별한 의미를 가짐

ex) `https://example.com/search?query=hello world` → `https://example.com/search?query=hello%20world`

- 비 ASCII 문자(한글, 일본어 등) 처리

URL은 ASCII 문자 집합(영어, 숫자, 일부특수문자)만 허용

ex) 한글 → `%ED%95%9C%EA%B8%80`

- 보안 및 일관성 유지

웹서버와 클라이언트 간 통신에서 일부 문자(<, >, " 등)는 보안 문제를 유발할 수 있어 인코딩을 통해 안전하게 전송

ex) `<script>alert(1);</script>` → `%3Cscript%3Ealert%281%29%3B%3Cscript%3E`

URL 인코딩이 필요한 문자 목록

인코딩이 필요한 문자

문자유형	예	인코딩된 값
공백		%20
예약문자(URL구조에서 사용)	? & # / :	%3F %26 %23 %2F %3A
특수문자	! @ \$ % + =	%21 %40 %24 %2B %3D
비ASCII 문자(한글, 특수기호..)	한 日	%ED%95%9C %E6%97%A5

안전한 문자(인코딩 불필요)

- 영문 대/소문자
- 숫자
- 일부 특수문자 (- _ . ~)

BurpSuite를 통해 BASE64 디코딩

1. 파이어폭스 주소창에 주소 입력

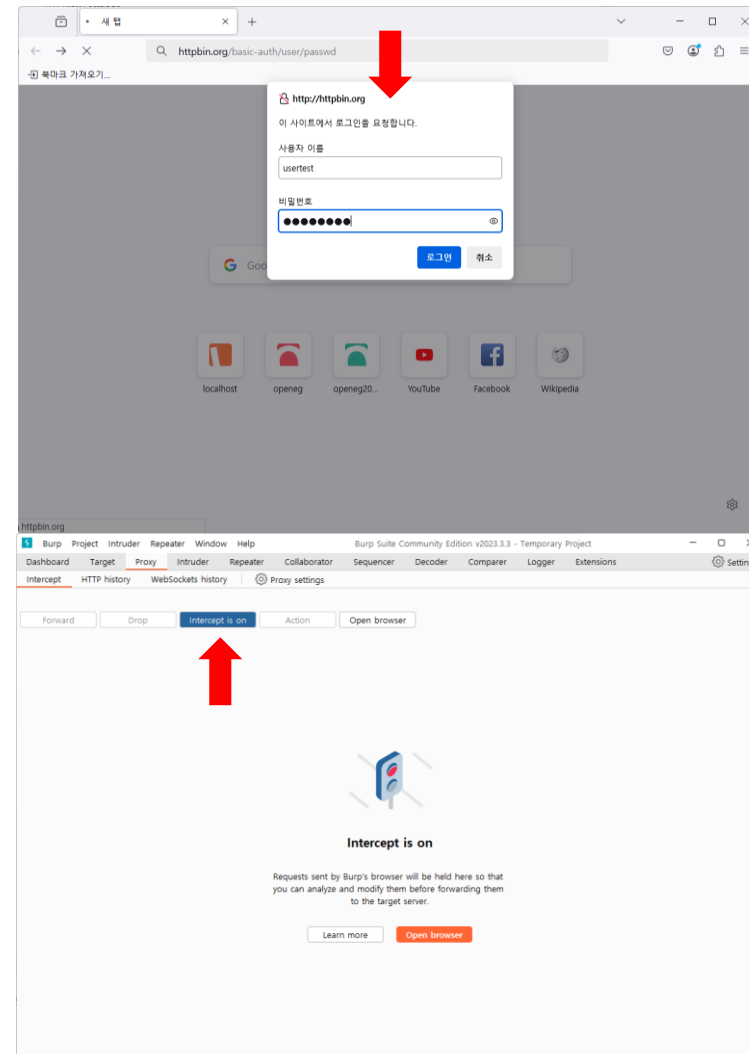
파이어폭스 주소창에 아래 주소 입력

<http://httpbin.org/basic-auth/user/passwd>

화면상에 로그인 창이 나타남

BurpSuite Proxy에서 [Intercept is off] 버튼을 눌러
[Intercept is on] 으로 전환

사용자 이름 및 암호에 아무 값이나 입력한 후 [확인] 버튼



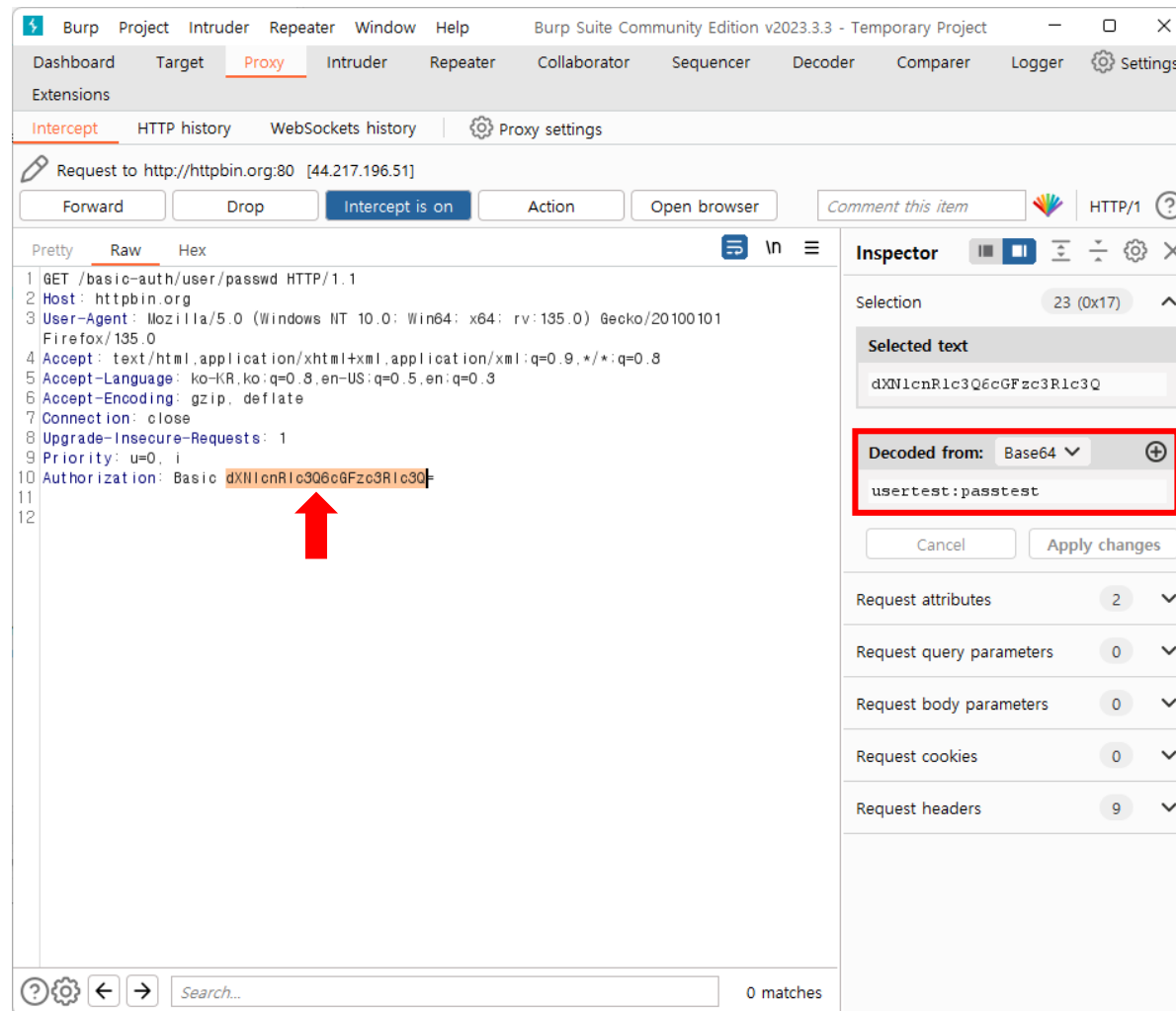
BurpSuite를 통해 BASE64 디코딩

2. BurpSuite Intercept 화면에 Request 내용이 보임
HTTP Request의 Header 내용 중 Authorization 내용에
BASE64 인코딩된 내용이 들어있음



BurpSuite를 통해 BASE64 디코딩

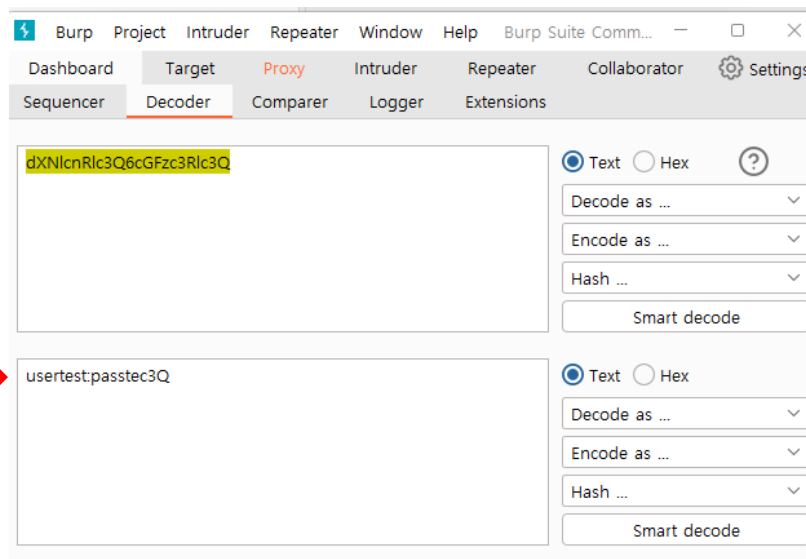
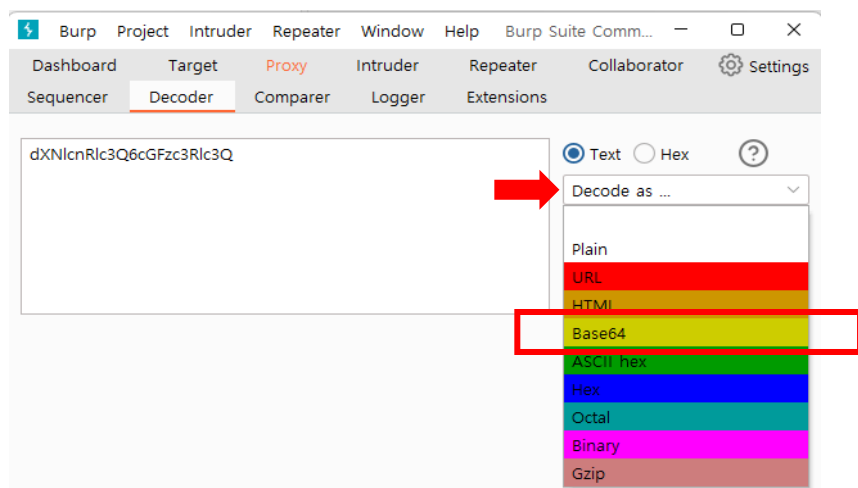
3. 해당 BASE64 인코딩 내용 선택하여 디코딩
BASE64 인코딩된 문자열을 더블클릭하면 우측에 Base64 디코딩된 내용을 볼 수 있음
해당 문자열을 CTRL-C (Copy) 함



BurpSuite를 통해 BASE64 디코딩

4. Decode 메뉴에서 디코딩

복사한 BASE64문자열을 입력창에 붙여 넣고 우측 'Decode as ...' 에서 Base64 선택
하단에 디코딩된 문자열이 출력됨



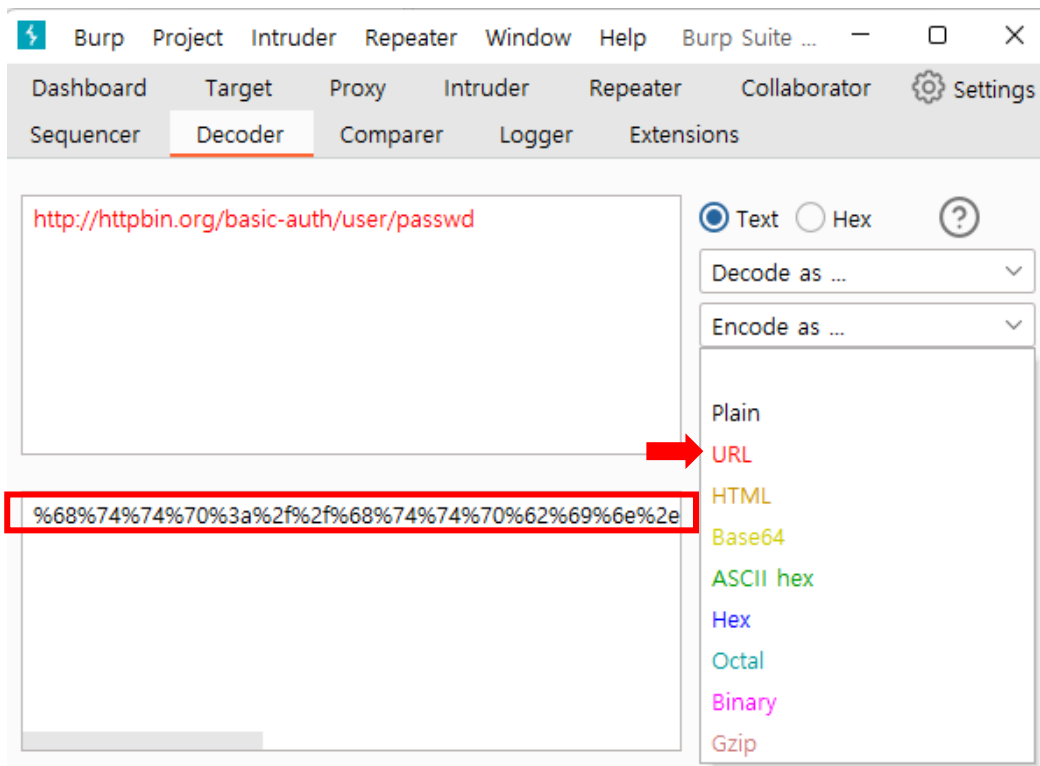
BurpSuite를 통해 URL인코딩/디코딩

1. 입력창에 사이트 주소 붙여 넣고 인코딩

앞서 접속했던 사이트 주소를 입력창에 넣고 'Encode as ...' 선택

종류 중 'URL' 선택

하단에 인코딩된 문자열이 출력됨

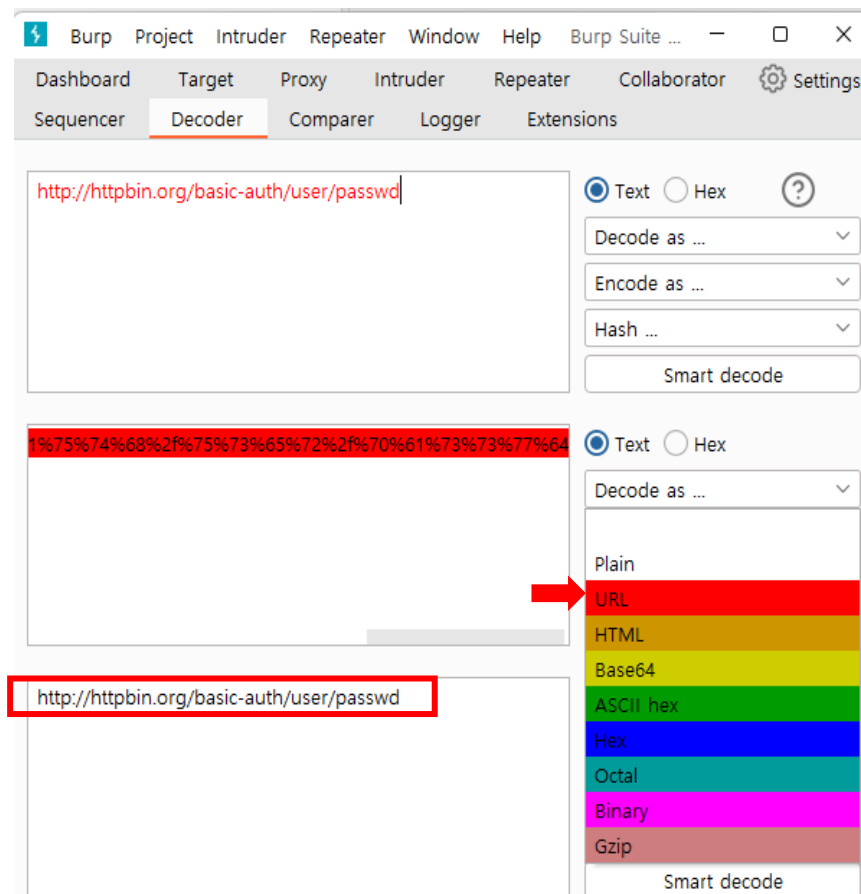


BurpSuite를 통해 URL인코딩/디코딩

2. 인코딩 된 문자열로 다시 URL 디코딩

앞서 접속했던 사이트 주소를 입력창에 넣고 'Encode as ...' 선택
종류 중 'URL' 선택

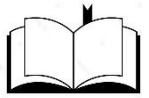
하단에 디코딩된 원본 문자열이 출력됨





요약 정리

- 실습환경은 Spring Boot 기반 웹 애플리케이션과 BurpSuite 프록시로 구성되어 있다.
- 웹 서비스는 브라우저, 웹 서버, WAS, DB 간의 통신 흐름으로 운영된다.
- HTTP는 요청과 응답 메시지로 통신하며, 상태 정보를 유지하지 않는다.
- HTTP 요청 안에는 바디 외에 메서드, 경로, 쿠키, 파라미터 같은 헤더 등이 들어있다.
- HTTP 응답 안에는 바디 외에 상태 코드가 들어있어, 서버의 처리 상태를 알 수 있다..
- BASE64와 URL 인코딩은 특수 문자나 바이너리 데이터를 안전하게 전달하기 위해 사용된다.



참고 문헌

- [KISA] 소프트웨어 개발 보안 가이드(2021.12.29)
- [KISA] 소프트웨어 보안약점 진단가이드(2021)
- [오픈이지박스] 실무에 바로 적용하는 해킹 방어를 위한 JAVA 시큐어코딩
- [홍릉과학출판사] 소프트웨어 개발 보안 실무 - 시큐어 코딩 원리와 실제
- [secu BOOK] 웹 해킹과 시큐어코딩 탐지/수정 실습가이드
- [에이콘출판사] 보안 위협 모델링 Threat Modeling - 위협 식별과 대응을 위한 소프트웨어 설계
- [책만] 모두를 위한 소프트웨어 보안 설계와 구현 - 취약점 분석부터 위협 모델링과 완화 전략, 보안 패턴, 암호화까지 안전한 소프트웨어의 모든 것
- [위키박스] 웹 애플리케이션 보안 완벽 가이드 - 체계적으로 배우는 안전한 웹 애플리케이션 제작 기법
- [한빛미디어] 웹 애플리케이션 보안 - 정찰, 공격, 방어 세 단계로 배우는 웹 애플리케이션 보안의 모든 것
- [한빛미디어] 웹 해킹 & 보안 완벽 가이드 - 웹 애플리케이션 보안 취약점을 겨냥한 공격과 방어
- SpringBoot - <https://spring.io/>
- Spring Initializ - <https://start.spring.io/>
- Postman - <https://www.postman.com/>

The background is a vibrant blue gradient. It is decorated with various abstract elements: a large sphere with a gradient in the top left, a smaller sphere with diagonal stripes in the top right, and another sphere with horizontal stripes in the bottom left. Several network-like structures, consisting of small circles connected by thin lines, are scattered across the image. There are also solid-colored rectangles and trapezoids in different shades of blue. The central text is in a bold, white, sans-serif font.

감사합니다.



퀴즈

1. 다음 중 웹 애플리케이션 서버 로직의 3대 계층에 속하지 않는 것은?

- 1) 데이터 계층
- 2) 네트워크 계층
- 3) 비즈니스 계층
- 4) 프레젠테이션 계층

2. 다음 중 HTTP에 대한 설명으로 옳은 것은?

- 1) 무상태 방식
- 2) UDP를 기본적으로 사용
- 3) Request Method로 GET, POST, PUT, REMOVE 등이 있음
- 4) Response Status Code로 2xx는 서버에 에러가 있다는 의미

3. 다음 중 BASE64 인코딩에 대한 내용으로 틀린 것은?

- 1) 이메일, JSON, XML 등 텍스트 전송 환경에서 안전하게 사용
- 2) 부족한 비트는 0으로 패딩 처리되며, =로 표현
- 3) 데이터 손상 및 인코딩 문제를 방지함
- 4) 데이터를 8비트 단위로 분할하며, 총 64개의 문자 집합으로 표현



퀴즈 정답

1. 다음 중 웹 애플리케이션 서버 로직의 3대 계층에 속하지 않는 것은?

2) 네트워크 계층

웹 애플리케이션 서버 로직은 프레젠테이션 계층, 비즈니스 계층, 데이터 계층으로 구성되어 있습니다.

2. 다음 중 HTTP에 대한 설명으로 옳은 것은?

1) 무상태 방식

HTTP는 주로 TCP기반 무상태 방식으로 통신하며, GET, POST, PUT, DELETE 등의 Request Method를 사용합니다. Response Status Code가 2xx인 경우는 요청이 성공적으로 이루어졌다는 의미입니다.

3. 다음 중 BASE64 인코딩에 대한 내용으로 틀린 것은?

4) 데이터를 8비트 단위로 분할하며, 총 64개의 문자 집합으로 표현

BASE64는 데이터를 6비트 단위로 분할합니다.